

JavaScript

Project Workbook
Student Edition

Presented By
LK LearnKey®

JavaScript

Project Workbook

LearnKey creates signature multimedia courseware. LearnKey provides expert instruction for popular computer software, technical certifications, and application development with dynamic video-based courseware and effective learning management systems. For a complete list of courses, visit <https://www.learnkey.com>.

All rights reserved. Unauthorized reproduction or distribution is prohibited.

© 2021 LearnKey
www.learnkey.com

Presented By
 **LearnKey®**

Table of Contents

Using This Workbook	6
Best Practices Using LearnKey's Online Training	7
Skills Assessment	8
JavaScript Video Times	9
Domain 1	
Fill-in-the-Blanks Domain 1	11
Using Assignment, Increment, and Decrement Operators	13
Addition and Subtraction	14
Multiplication, Division, and Modulus Operators	15
Compound Assignment Operators	16
Comments, Indentations, and Naming Conventions	17
Using Script and Noscript Tags	18
Constants and Reserved Keywords	19
The Debugger Keyword	20
Using Breakpoints and Console.log	21
The Use of Inline and External Scripts	22
Implementing Exception Handling	23
Managing States	24
Display Dialogs	25
Determining Screen Size	26
Domain 2	
Fill-in-the-Blanks Domain 2	28
Number, Boolean, and String Variables	30
Null and Undefined Variables	31
Type of Operators and Type Checking Functions	32
The Use Strict Phrase	33
Converting Between Data Types	34
Formatting Numbers	35
String Operations	36
Initialization	37
Build and Iterate Through Arrays	38
Sorting and Searching	39
Push and Pop	40
Shift and Unshift	41
Access an Element	42
Using Multi-Dimensional Arrays	43
Properties and Methods	44
Instantiation	45
Date Object and Time	46
Retrieving Date Parts	47
Localization	48

Add and Subtract Dates	49
Random Numbers	50
Rounding and Absolute Methods	51
Floor and Ceiling	52
Min and Max	53
Powers and Square Roots	54
Reusable Code	55
Scopes and Variable Definitions	56
Value vs. Reference	57
Return Values	58

Domain 3

Fill-in-the-Blanks Domain 3	60
Equals and Not Equals	61
Greater Than or Less Than	62
Other Comparison Operators	63
Not	64
And vs. Or	65
If-Else	66
Else If	67
Switch	68
Nested If	69
For	70
Do and While Loops	71
Break and Continue	72

Domain 4

Fill-in-the-Blanks Domain 4	74
Window	75
Document, Body, and Other HTML Elements	76
Onchange	77
Onmouseover	78
Onload and Onclick	79
Onmouseout	80
Onkeydown	81
InnerHTML	82
Document.write	83
GetElement	84
SetAttribute and CreateElement	85

Domain 5

Fill-in-the-Blanks Domain 5	87
Retrieving Form Values	88
DOM Path	89
Values from Elements	90
Prepopulating Values	91
Masking Values	92

Uppercase and Lowercase	93
Validation	94
NaN	95
Onsubmit	96
Post vs. Get and Submission Targets	97

Appendix

Glossary	99
JavaScript Suggested Study Plan	102
JavaScript Domain 1 Outline	115
JavaScript Domain 2 Outline	116
JavaScript Domain 3 Outline	117
JavaScript Domain 4 Outline	118
JavaScript Domain 5 Outline	119

Using This Workbook

This project workbook contains practice projects and exercises to reinforce the knowledge you have gained through the video portion of the **JavaScript** course. The purpose of this workbook is twofold. First, get you further prepared to pass the JavaScript exam, and second, to get you job-ready skills and increase your employability in the area of web application development using JavaScript.

The projects within this workbook follow the order of the video portion of this course. To save your answers in this workbook, you must first download a copy to your computer. You will not be able to save your answers in the web version. You can complete the workbook exercises as you go through each section of the course, complete several of these at the end of each domain, or complete them after viewing the entire course. The key is to go through these projects to strengthen your knowledge of this course.

Each project is based upon a specific video (or videos) in the course and specific test objectives. The materials you will need for this course include:

- Visual Studio Code software. You will need to configure Visual Studio Code to work with JavaScript. Use this link: [Working with JavaScript in Visual Studio Code](#)
- The practice files for these projects.
- One or more web browsers to use to test your webpages. The course uses Microsoft Edge, but you should use at least two browsers (Google Chrome, Firefox, Safari, Internet Explorer) for best testing results.

For Teachers

LearnKey is proud to provide extra support to instructors upon request. For your benefit as an instructor, we also provide an instructor support .zip file containing answer keys, completed versions of the workbook project files, and other teacher resources. This .zip file is available within your learning platform's admin portal.

Notes

- Extra teacher notes, when applicable, are in the Project Details box within each exercise.
- Exam objectives are aligned with the course objectives listed in each project, and project file names correspond with these numbers.
- The Finished folder in each domain has reference versions of each project. These can help you grade projects.
- Short answers may vary but should be similar to those provided in this workbook.
- Teachers may consider asking students to add their initials, student ID, or other personal identifiers at the end of each saved project.
- Refer to your course representatives for further support.

We value your feedback about our courses. If you have any questions, comments, or concerns, please let us know by visiting <https://about.learnkey.com>.

Best Practices Using LearnKey's Online Training

LearnKey offers video-based training solutions that are flexible enough to accommodate private students and educational facilities and organizations.

Our course content is presented by top experts in their respective fields and provides clear and comprehensive information. The full line of LearnKey products has been extensively reviewed to meet superior standards of quality. Our course content has also been endorsed by organizations such as Certiport, CompTIA®, Cisco, and Microsoft. However, it is the testimonials given by countless satisfied customers that truly set us apart as leaders in the information training world.

LearnKey experts are highly qualified professionals who offer years of job and project experience in their subjects. Each expert has been certified at the highest level available for their field of expertise. This expertise provides the student with the knowledge necessary to obtain top-level certifications in their chosen field.

Our accomplished instructors have a rich understanding of the content they present. Effective teaching encompasses presenting the basic principles of a subject and understanding and appreciating organization, real-world application, and links to other related disciplines. Each instructor represents the collective wisdom of their field and within our industry.

Our Instructional Technology

Each course is independently created based on the manufacturer's standard objectives for which the course was developed.

We ensure that the subject matter is up-to-date and relevant. We examine the needs of each student and create training that is both interesting and effective. LearnKey training provides auditory, visual, and kinesthetic learning materials to fit diverse learning styles.

Course Training Model

The course training model allows students to undergo basic training, building upon primary knowledge and concepts to more advanced application and implementation. In this method, students will use the following toolset:

Pre-assessment: The pre-assessment is used to determine the student's prior knowledge of the subject matter. It will also identify a student's strengths and weaknesses, allowing the student to focus on the specific subject matter he/she needs to improve the most. Students should not necessarily expect a passing score on the pre-assessment as it is a test of prior knowledge.

Video training sessions: Each training course is divided into sessions or domains and lessons with topics and subtopics. LearnKey recommends incorporating all available external resources into your training, such as student workbooks, glossaries, course support files, and additional customized instructional material. These resources are located in the folder icon at the top of the page.

Exercise labs: Labs are interactive activities that simulate situations presented in the training videos. Step-by-step instructions and live demonstrations are provided.

Post-assessment: The post-assessment is used to determine the student's knowledge gained from interacting with the training. In taking the post-assessment, students should not consult the training or any other materials. A passing score is 80 percent or higher. If the individual does not pass the post-assessment the first time, LearnKey recommends incorporating external resources, such as the workbook and additional customized instructional material.

Workbook: The workbook has various activities, such as fill-in-the-blank worksheets, short answer questions, practice exam questions, and group and individual projects that allow the student to study and apply concepts presented in the training videos.

Skills Assessment

Instructions: Rate your skills on the following tasks from 1-5 (1 being needs improvement, 5 being excellent).

Skills	1	2	3	4	5
Complete or debug code that uses assignment and arithmetic operators.					
Apply JavaScript best practices.					
Evaluate the use of inline and external scripts.					
Implement exception handling.					
Complete and debug code that interacts with the Browser Object Model (BOM).					
Declare and use variables of primitive data types.					
Declare and use arrays.					
Complete and debug code that uses objects.					
Complete and debug code that uses built-in math functions.					
Complete and debug a function that accepts parameters and returns a value.					
Evaluate expressions that use logical and comparison operators.					
Complete and debug decision statements.					
Complete and debug loops.					
Identify and construct the DOM (Document Object Model) tree.					
Identify and handle HTML events.					
Complete and debug code that outputs to an HTML document.					
Complete and debug code that locates, modifies, and adds HTML elements and attributes.					
Complete and debug code that retrieves input from forms and sets form field values.					
Complete and debug code to perform input validation.					
Describe the form submission process.					

JavaScript Video Times

Domain 1	Actual Time
Introduction	00:02:47
Assignment and Arithmetic Operators	00:16:28
JavaScript Best Practices	00:17:19
Inline and External Scripts	00:05:37
Exception Handling	00:03:45
Browser Object Model Interaction	00:07:28
Total Time	00:53:24

Domain 2	Actual Time
Primitive Data Types	00:25:37
Declare and Use Arrays	00:13:51
Complete and Debug Code on Objects	00:15:49
Built-In Math Functions	00:11:36
Parameters and Return Values	00:13:44
Total Time	01:20:37

Domain 3	Actual Time
Logical and Comparison Operators	00:13:44
Decision Statements	00:09:18
Loops	00:13:02
Total Time	00:36:04

Domain 4	Actual Time
DOM Tree	00:06:49
HTML Events	00:12:27
Output to HTML Documents	00:03:16
Locate, Modify, and Add Elements	00:13:42
Total Time	00:36:34

Domain 5	Actual Time
Retrieve Input and Set Form Field Values	00:09:51
Input Validation	00:08:56
Form Submission Process	00:05:40
Conclusion	00:02:24
Total Time	00:26:51

JavaScript

Domain 1

Presented By
LK LearnKey®

Fill-in-the-Blanks Domain I

Instructions: While watching Domain I, fill in the missing words according to the information presented by the instructor.
[References are found in the brackets.]

Assignment and Arithmetic Operators

1. JavaScript has a _____ sign which means, is this equal to this value? [Assignment]
2. In JavaScript, ++ means _____ one to this. [Increment and Decrement]
3. With addition, the plus sign is used just as it is on a _____. [Addition]
4. When using JavaScript, _____ your work after every one or two changes. [Subtraction]
5. When working with division in JavaScript, the remainder is otherwise known as the _____. [Division]
6. In JavaScript the method to control the number after the decimal point is _____. [Multiplication]
7. The modulus symbol in JavaScript is the _____ symbol. [Modulus]
8. Compound assignment operators add, subtract, multiply, divide, and _____ values at the same time. [Compound Assignment Operators]

JavaScript Best Practices

9. Comments come in two forms: _____ comments, and _____ comments. [Comments]
10. A JavaScript best practice is to indent _____ inside of a single function. [Indentations]
11. For naming conventions, _____ casing is a JavaScript best practice. [Naming Conventions]
12. _____ are pop-up messages that appear in a web browser. [Noscript]
13. Variables can change throughout their use in code but _____ cannot. [Constants]
14. In JavaScript, reserved keywords cannot be used to name a _____. [Reserved Keywords]
15. You can use the debugger keyword to stop the _____ in progress. [Debugger Keyword]
16. Breakpoints allow you to _____ code, check on variables, and then run code step by step to continue processing it. [Set Breakpoints]
17. You will often have as a debugging tool a console in which you can enter _____ to get information. [Console.log]

Inline and External Scripts

18. Like style sheets, _____ are better managed when they are external. [When to Use]
19. As a best practice, JavaScript scripts should be placed into a _____ folder. [How to Use]
20. Anything used on an _____ style sheet should not be repeated inline. [Use Both Scripts]

Exception Handling

21. One unique aspect of JavaScript is that _____ messages do not typically pop up in a web browser when they occur. [Try and Catch]
22. A finally block will run no matter which part, _____ or _____, runs. [Finally]

Browser Object Model Interaction

23. With HTML, CSS, and JavaScript, applications are built with many opportunities for _____. [Manage State]
24. Alerts are basically _____ displayers and have an OK button. [Display Dialogs]
25. _____ is a browser object and width and height are two of its properties. [Determine Screen Size]

Using Assignment, Increment, and Decrement Operators

One of the most important aspects to understand about JavaScript is the proper use of operators. The first two projects in this workbook will cover some of these operators in detail. The first three operators you will work with are the assignment, increment, and decrement operators. Increment operators tell a variable to increase by one and decrement operators tell a variable to decrease by one.

Note: For all projects in this course, when you are asked to open a file, assume you need to open it in your code editor unless you see a different specification, like opening the file in your web browser.

Purpose:

After completing this project, you will know how to use assignment, increment, and decrement operators.

Steps for Completion:

1. Open the **I1a-cartassignments.htm** file from the Music I Student folder to your code editor.
2. Find this comment: `<!-- Add the three operators for the three input buttons-->` approximately 60 lines down in the document.
3. Find the three input buttons with IDs of `manqtyadd`, `manqtysub`, and `manqty zero`, respectively.
4. On the line with the button with the ID of `manqtyadd`, add the increment operator to the value property.
5. On the line with the button with the ID of `manqtysub`, add the decrement operator to the value property.
6. On the line with the button with the ID of `manqtyadd`, add the necessary operator and value to set the `manqty` element to 0.
7. Save your changes.
8. Preview the webpage in a web browser. You should be able to click the +, -, and 0 buttons and have the value in the text box increase by one, decrease by one, and reset to 0 respectively.

Project Details

Project File:

I1a-cartassignments.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain I

Topic: Assignment and Arithmetic Operators

Subtopics: Assignment; Increment and Decrement

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.1 Complete and debug code that uses assignment and arithmetic operators

I.1.a Assignment

I.1.b Increment

I.1.c Decrement

Addition and Subtraction

JavaScript, like many other programming languages, uses the computer standard of a + for addition, a – for subtraction, a * for multiplication, and a / for division. The key point of knowledge for the exam is to know which operators to use given a situation or problem to solve. That is exactly what you will practice in this and the next project.

Purpose:

After completing this project, you will be able to identify which operators to use in a given situation.

Steps for Completion:

1. Open the **IId-cart-operators.htm** file from the MusicI Student folder in a web browser.
2. Take note of what needs to be added to get to a subtotal and then subtracted to get to a grand total.
3. Open the **IId-cart-operators.htm** file from the MusicI Student folder in your code editor.
4. Find this comment: `<!-- Add the addition and subtraction operators in the correct places -->`
5. Add addition operators and subtraction operators in the correct places in the Update Cart button script to get the correct subtotal and grandtotal, ignoring the sales tax for now.
6. Save your changes.
7. Preview the webpage in a web browser.
8. Click the Update Cart button. You should see a total of 799.97.

Project Details

Project Files:

IId-cart-operators.htm

Estimated Completion

Time:

10 minutes

Video Reference:

Domain I

Topic: Assignment and Arithmetic Operators

Subtopics: Addition; Subtraction; Multiplication

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.I Complete and debug code that uses assignment and arithmetic operators

I.I.d Addition

I.I.e Subtraction

Multiplication, Division, and Modulus Operators

The next three operators we are going to look at in some detail are the multiplication operator, which is a `*` in computing terms, division operator, which is a `/` in computing terms, and the modulus. A modulus, as indicated by a `%`, indicates a remainder. For example, in terms of integers, $10/3=3$. $10\%3=1$. Why? $10/3=3$ with a remainder of 1. That 1 is the modulus. You need to know this for the exam and you will need this in real life for when you need to calculate a remainder, as you will do in this project, which calculates the number of pieces needed for soundproofing a room at 25 square feet per piece.

Purpose:

Upon completing this project, you will know how and when to use the multiplication, division, and modulus operators.

Steps for Completion:

1. Open the **IIf-soundproofing.htm** file from the MusicI Student folder.
2. Look for this comment about 45 lines down the document: `<!-- Three operators needed below -->`
3. Over the next seven lines, there are three `//` that need replacing with the proper operators. Replace them with the correct operators so that the following takes place:
 - a. The correct number of pieces needed are calculated given that each piece covers 25 square feet.
 - b. The number of square feet for the final piece is indicated.
 - c. The total cost, given each piece sells for \$20, is displayed.
4. Save your changes.
5. Preview the file in a web browser.
6. Click in the Enter the Square Footage field.
7. Type: **460**
8. Click the Calculate button. If you get a message saying you need 19 pieces and there will be 10 square feet left and that the total cost is \$380, congratulations. If not, go back and look at the logic you used for the operators and try again.

Project Details

Project Files:

IIf-soundproofing.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain I

Topic: Assignment and Arithmetic Operators

Subtopic: Multiplication; Division; Modulus

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.I Complete and debug code that uses assignment and arithmetic operators

I.I.f Division

I.I.g Multiplication

I.I.h Modulus

Compound Assignment Operators

Compound assignment operators can add, subtract, multiply, divide, and update values at the same time. Take, for example, this equation: $x = x + 5$. With a compound operator, that same equation can be written this way: $x += 5$. Compound operators $-=$, $*=$, and $/=$ subtract, multiply, and divide values, respectively.

Purpose:

After completing this project, you will know how to use a compound operator to update a value.

Steps for Completion:

1. Open the **I li-cart-compound.htm** file in your Music I Student folder to your code editor.
2. Scroll to the bottom of the page and find the `<script>` section.
3. Within the script, find the `// add five` comment.
4. Replace the `// add five` comment with a compound operator assignment to add five to the `newQty` variable.
5. Replace the `// subtract five` comment with a compound operator assignment to subtract five from the `newQty` variable.
6. Save your changes.
7. Preview the file in a web browser.
8. Click the Add Five button. It should add five to the quantity amount for the mandolin.
9. Click the Subtract Five button. It should subtract five from the quantity amount for the mandolin.
10. Close your open files in both your web browser and code editor.

Project Details

Project Files:

I li-cart-compound.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain I

Topic: Assignment and Arithmetic Operators

Subtopic: Compound Assignment Operators

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.I Complete and debug code that uses assignment and arithmetic operators

I.I.i Compound assignment operators ($+=$, $-=$, $*=$, $/=$, $%=$)

Comments, Indentations, and Naming Conventions

Part of writing good code in any programming language is to make the code readable, consistent, and, as much as possible, easy for someone else reading the code to discern what is going on with the code. To that end, we should add comments to code when it becomes necessary to explain what is going on in code. A `//` within a script indicates a single-line comment. Text within `/*` and `*/` symbols indicate a multiline comment.

Code inside of functions, if statements, and similar methods should be indented, so that one can easily understand which code block is inside of which statement. In addition, when naming objects in JavaScript, the usual best practice is to use camel casing. Camel casing involves a lowercase first word and then the capitalization of the first letter of each subsequent word in an object. For example, a variable named total quantity could be named as `totalQty` or `totalQuantity` using camel casing.

Purpose:

After completing this project, you will know how to implement comments, indentations, and naming conventions within JavaScript code.

Steps for Completion:

1. Open the **I2a-soundproofing.htm** file from your Music I Student folder.
2. Scroll down about 45 lines to find this code: `<!-- Code cleanup needed in the next paragraph -->`
3. Between the `<p>` and `</p>` tags below the comment, indent the code so that it is even and readable.
4. Click in the blank line above this code: `document.getElementById('sqftremaining').innerHTML = 'The last piece will take up ' + parseInt(document.getElementById('sqft').value % 25) + ' square feet.'`
5. Add a single-line comment explaining that the modulus operator is being used in the line below.
6. On the line below where you just added the comment, rename the `sqftremaining` object to use camel casing.
7. Find the code `<p id="sqftremaining">` several lines below the current line.
8. Rename the `sqftremaining` object to use the same camel casing you used in Step 6.
9. Save your changes.
10. Preview the webpage in a browser.
11. To make sure the webpage still works, click in the Enter the square footage field and type **490**
12. Click the Calculate button. You should see that 20 pieces are needed, and 15 square feet will remain.

Project Details

Project Files:

I2a-soundproofing.htm

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain I

Topic: JavaScript Best Practices

Subtopics: Comments; Indentations; Naming Conventions

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.2 Apply JavaScript best practices

I.2.a Comments

I.2.b Indentations

I.2.c Naming conventions

Using Script and Noscript Tags

As we continue learning the basics of JavaScript code, we want to know when to use a script and a noscript tag. Script tags are used to hold JavaScript code and keep it away from HTML code on a webpage. As an example, functions are often defined within script tags and then called from elsewhere on a webpage.

A noscript tag has one purpose: to alert a user that the browser being used does not support JavaScript. This could stem from a very old browser being in use, but more likely the cause is a browser that has JavaScript disabled. Either way, we want to let a user know when JavaScript is not working. The noscript tag accomplishes that.

Purpose:

After completing this project, you will know how to utilize the script and noscript tags within a webpage.

Steps for Completion:

1. Open the **I2d-tuningkits.htm** file from your Music I Student folder.
2. Scroll to the end of the document.
3. Click on a blank line just inside the closing body tag.
4. Add an opening and closing script tag.
5. Within the new script tag, add an alert with the text, **Your browser supports JavaScript**
6. Below the closing script tag, add an opening and closing noscript tag.
7. Within the noscript tag, add this text, **Your browser does not support JavaScript**
8. Save your changes.
9. Preview the file in a web browser. Either you will see that your browser supports JavaScript, or, you will see that it does not support JavaScript.

Project Details

Project Files:

I2d-tuningkits.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain I

Topic: JavaScript Best Practices

Subtopic: Noscript

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.2 Apply JavaScript best practices

I.2.d Noscript

Constants and Reserved Keywords

Variables, which are containers that store information that we can reuse in code, will be covered in detail later in this course. One type of variable we will cover now, though, is that of a constant. A constant is a variable that cannot have its value changed once it is set. This is helpful for when you want to use something such as a rate for calculations throughout a webpage.

When building variables, functions, or anything else with a name, you will want to avoid using reserved keywords, that is, keywords already in use in JavaScript, as an object name. Pay attention to keywords you see in use throughout this course as you could be asked on the exam which words are keywords and which ones are not.

Purpose:

After completing this project, you will know how to declare constants and you will know some of the more prevalent reserved keywords in JavaScript.

Steps for Completion:

1. Open the **12e-cart-constants.htm** file from the Music I Student folder.
2. Scroll down to the end of the document.
3. On the line before the closing body tag, add an opening and closing script tag.
4. Within the new script tag, create a blank line if necessary.
5. On the blank line, within the script tag, add a constant variable named **salesTaxRate** and set the value to **.07**
6. Scroll up and find the following code: `<!-- sales tax rate constant goes in this area -->`
7. Look several lines below the comment. Find this code: `document.getElementById('salesTax').innerHTML = parseFloat(document.getElementById('subtotal').innerHTML * .07).toFixed(2);`
8. In the code you just found, replace the `.07` value with the new `salesTaxRate` constant.
9. Save your changes.
10. Preview the page in a web browser.
11. Click the Update Cart button. Your total should be **856.67**.
12. For each declaration of a variable, indicate whether the declaration is valid or invalid due to the use of a reserved keyword:
 - a. `stepOne = "Sign up for a class."`
 - b. `function = "Sign up for a class"`
 - c. `void = null`
 - d. `emptySet = null`

Project Details

Project Files:

12e-cart-constants.htm

Estimated Completion Time:

15 minutes

Video Reference:

Domain I

Topic: JavaScript Best Practices

Subtopics: Constants; Reserved Keywords

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.2 Apply JavaScript best practices

I.2.e Constants

I.2.f Reserved keywords

The Debugger Keyword

Part of writing good code is knowing the tools you have to troubleshoot code and using those tools well. JavaScript does not have as many troubleshooting tools as do other languages, but, there are still a few good ones you can use, mostly within a web browser. If your code editor has troubleshooting tools, you can use those as well. One common troubleshooting tool to use is the debugger keyword. This will make the code pause at that point and then, within a browser, you can use the console within the browser to run tests on current variables and HTML elements.

Purpose:

After completing this project, you will have familiarity with the debugger statement and know how to use it within a webpage.

Steps for Completion:

1. Open the **l2g-cart-constants.htm** file from the MusicI Student folder.
2. Scroll down about 175 lines to this code: `// put the debugger here`
3. Replace the comment and code with the debugger keyword.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Press the F12 key to open a console window.
7. Click the Update Cart button. The code will pause at the point of the debugger and the console will appear.
8. Type: **`document.getElementById('subtotal').innerHTML`**
9. Press the Enter key. You should see a total of 809.97.
10. Find the Play symbol in the debugger and click it. The code will finish running.

Project Details

Project File:

l2g-cart-constants.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain I

Topic: JavaScript Best Practices

Subtopic: Debugger Keyword

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.2 Apply JavaScript best practices

I.2.g Debugger keyword

Using Breakpoints and Console.log

While the debugger statement is useful when you always want a breakpoint in code, you may want to apply breakpoints as code is running in the browser. This can be done on webpages that await user interaction, like the click of a button. You can use the console to insert breakpoints and then use those breakpoints to test values within a webpage.

The console.log feature allows you to type elements and variables within the console and receive a return of the value of the element or variable you type.

Purpose:

After completing this project, you will know how to use breakpoints and the console.log statement. Note that the steps will vary based on the browser you are using.

Steps for Completion:

1. Open the **12h-cart-constants.htm** file from the MusicI Student folder.
2. Preview the webpage in a web browser.
3. Press the F12 key to open a console window.
4. In the Sources area, select the file.
5. Expand the Event Listener Breakpoints area.
6. Expand the Mouse area.
7. Select the click checkbox.
8. Click the Update Cart button.
9. Notice, in the debugger, the Watches window. Scroll through the Watches window to see values for the HTML elements on the page.
10. Play the rest of the code.
11. Refresh the webpage.
12. Click the Update Cart button. The code should stop at the breakpoint you added earlier.
13. Click the Console tab.
14. Type a console.log statement for the innerHTML value of the subtotal element. It should be undefined.
15. Step into the code.
16. Rerun the console.log statement you typed earlier. It should show a value of 809.97.
17. Close all your open webpages in your browser and files inside of your code editor.

Project Details

Project File:

12h-cart-constants.htm

Estimated Completion Time:

15 minutes

Video Reference:

Domain I

Topic: JavaScript Best Practices

Subtopic: Debugger Keyword; Set Breakpoints; Console.log

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.2 Apply JavaScript best practices

I.2.h Setting breakpoints

I.2.i Console.log

The Use of Inline and External Scripts

When building scripts with JavaScript code (or any script for that matter), one important decision to make is whether to put the script inline, meaning inside of a file, or in an external file. You also, both for the exam and beyond, will want to know the effects of having both inline and external scripts, especially when the same variable name is declared in both files.

Purpose:

After completing this project, you will have a better understanding of when to use scripts inline and when to use external scripts. You will also know how variables behave when they are declared in both inline and external scripts.

Steps for Completion:

1. Open the **I3a-cart-compound.htm** file from the Music I Student folder in a web browser.
2. View the HTML source of the file. Is the JavaScript code inline or in an external file?
3. Give two reasons in favor of using external scripts over inline scripts:
4. Click the Update Cart button. The total amount should be 856.67.
5. In your code editor, open the I3-cart-compound.htm file and the main.js files from your Music I Student folder.
6. Notice the salesTaxRate of .075 within the main.js file.
7. Return to the I3-cart-compound.htm file and scroll to the script tag at the bottom of the page.
8. Notice the salesTaxRate of .07. This is the rate used in the calculation to create a total amount of 856.67.
9. In the head section, add a tag that links the current page to the main.js file in the scripts folder.
10. Save your changes.
11. Preview the webpage in a web browser.
12. Click the Update Cart button. What is the total amount?
13. What caused the amount to change?

Project Details

Project Files:

I3a-cart-compound.htm
main.js

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain I

Topic: Inline and External Scripts

Subtopics: When to Use; How to Use; Use Both Scripts

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.3 Evaluate the use of internal and external scripts

I.3.a When to use

I.3.b How to use

I.3.c What happens when scripts are used at multiple levels

Implementing Exception Handling

Exceptions are errors that happen while code is running. Unless handled properly, exceptions will usually cause an app to crash. To handle these exceptions more gracefully, code that could fail during runtime should be implemented using a try/catch/finally block, to where the try block contains the code one attempts to run, the catch block runs if the code in the try block causes an exception, and the finally block contains code that runs whether the try block worked or did not work.

Purpose:

After completing this project, you will know how to use the try, catch, and finally keywords in JavaScript code.

Steps for Completion:

1. Open the **I4a-practice.htm** file from your Music I Student folder.
2. Locate the script tag about 20 lines down the page.
3. In the script tag, wrap the current function inside of a try block.
4. Below the try block, add a catch block that will catch any possible error in the code and, if there is an error, display an alert with the message, **Error:**, plus the error's description.
5. Below the catch block, add a finally block that displays an alert with the message, **Thank you for visiting our site**
6. Save your changes.
7. Preview the webpage in a web browser.
8. Enter a number amount in the available text field.
9. Press the Tab key. You should see an error message stating that something is undefined and the thank you message for visiting the site.
10. Return to your code editor.
11. Within the try block, change the text `document.getElementById` to **document.getElementById** as ID is the improper casing for the `getElementById` method.
12. Save your changes.
13. Refresh the webpage in the web browser.
14. Enter a number amount in the available text field.
15. Press the Tab key. You should see a calculation on-screen and the thank you message for visiting the site.

Project Details

Project Files:

I4a-practice.htm

Estimated Completion Time:

15 minutes

Video Reference:

Domain I

Topic: Exception Handling

Subtopic: Try and Catch; Finally

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.4 Implement exception handling

I.4.a Try

I.4.b Catch

I.4.c Finally

Managing States

A state of a JavaScript object describes the current mode of an object. Windows can be open or closed. Objects can be made to be visible or invisible. Buttons can be enabled or disabled. For the exam, make sure you understand the meaning of a state of an object. In the upcoming project, you will get a small sample of managing states.

Purpose:

After completing this project, you will have a better understanding of the concept of managing states for JavaScript objects.

Steps for Completion:

1. Open the **I5a-cart-checkout.htm** file from the MusicI Student folder.
2. Scroll down to the script tag near the bottom of the document and find this code: // Close the checkWin window here
3. Replace the code and the comment with code to close the checkWin window.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Click the Checkout button. A new, small browser window will open. This window is now in the open state.
7. Click the Close Checkout button. The small window will close, putting the window in the closed state.

Project Details

Project Files:

I5a-cart-checkout.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain I

Topic: Browser Object Model Interaction

Subtopic: Manage State

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.5 Complete and debug code that interacts with the Browser Object Model (BOM)

I.5.a Display dialogs

Display Dialogs

Dialog boxes can capture user input, help a user confirm a step made on a webpage, or display an alert either for training purposes or when an error occurs. Specifically, there are three types of dialog boxes you need to know about for the exam:

Alert: displays a pop-up message on the screen. This does not take user interaction, except for displaying an OK button.

Confirm: an alert-like dialog with an OK and a Cancel button. Action can be coded based on which button a user clicks.

Prompt: a dialog which accepts user input. This input is usually then stored in a variable.

Purpose:

After completing this project, you will know which display dialog to use in given situations.

Steps for Completion:

1. Open the **15b-cart-welcome.htm** file from your Music I Student folder.
2. Scroll down to the script tag near the bottom of the document. Look for this comment: // enter the alert, confirm, and prompt keywords to replace the comments in the welcome function
3. Within the welcome function, there are three comment symbols. Replace each of the comment symbols with the correct type of display dialog (alert, confirm, or prompt), using the code as your guide. Each display dialog will be used once.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Enter your name in the prompt dialog box.
7. Click the OK button.
8. Click the OK button again to confirm your name.
9. Click the OK button to close the alert.

Project Details

Project Files:

15b-cart-welcome.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain I

Topic: Browser Object Model Interaction

Subtopic: Display Dialogs

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.5 Complete and debug code that interacts with the Browser Object Model (BOM)

I.5.a Display dialogs

Determining Screen Size

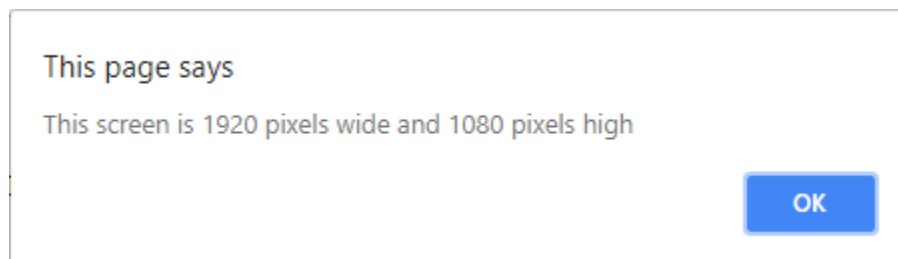
The final project in this domain will cover the code necessary to display the screen width and the screen height. This could be important information to know for developing an app as you may want to, for example, change a style sheet being used based on a screen size. The screen width and height are both properties of the screen object.

Purpose:

After completing this project, you will know how to obtain the screen width and screen height through JavaScript.

Steps for Completion:

1. Open the **15c-screensize.htm** file from the Music I Student folder.
2. Find the body tag, located about eight lines down in the file.
3. In the alert method for the onload event, enter the code necessary to display the screen width and the screen height, along with text describing the screen width and screen height.
4. Save your changes.
5. Preview the webpage in a web browser. You should see a message similar to the following:



6. Click the OK button.
7. Restore your browser to a different size.
8. Refresh the webpage. Does the screen size change?
9. Close your open files in both your web browser and in your code editor.

Project Details

Project File:

15c-screensize.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain I

Topic: Browser Object Model Interaction

Subtopic: Determine Screen Size

Objectives Covered:

I JavaScript Operators, Methods, and Keywords

I.5 Complete and debug code that interacts with the Browser Object Model (BOM)

I.5.b Determine screen size

JavaScript

Domain 2

Presented By
LK LearnKey®

Fill-in-the-Blanks Domain 2

Instructions: While watching Domain 2, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

Primitive Data Types

1. A _____ is a container used to hold information. [Number]
2. Boolean variables store two possibilities: _____ or _____. [Boolean]
3. Strings are variables that can store pretty much _____. [String]
4. Null means _____, not zero. [Null]
5. An undefined variable is one that has been _____, but has never had a value. [Undefined]
6. If you have a variable that is not acting the way you think it should, check out its _____. [typeof Operator]
7. Any number variable is a _____ decimal in JavaScript. [Type Checking Functions]
8. The _____ keyword tells JavaScript this is a variable and not an object. [Use Strict]
9. In many cases, a data type has to be explicitly _____ before it can be used. [Convert Between Data Types]
10. One common wish in JavaScript is to convert a number to _____. [Format Numbers]
11. JavaScript has many different ways in which strings can be _____. [String Operations]
12. One area that JavaScript is not that picky about is in the use of _____ marks. [Nesting Quotes]
13. If you know what a variable should _____ when you declare it, initialize it. [Initialization]

Declare and Use Arrays

14. Think of an array as having a _____ but with multiple sections, with each section storing one or more items. [Build and Define an Array]
15. When working with an array, you may want to show one item at a time in the form of a _____. [Iteration and Length]
16. Arrays that are coded but not filled in are considered _____. [Initialization]
17. The _____ is a great way to experiment with array features. [Sorting and Searching]
18. _____ adds an item to an array while _____ removes the last item from an array. [Push and Pop]
19. Shift really is the opposite of _____, and unshift really is the opposite of _____. [Shift and Unshift]
20. The first item in an array has an index number of _____. [Access an Element]

21. The _____ called multiArray starts a new array. [Understand Multi-Dimensional Arrays]

Complete and Debug Code on Objects

22. Objects are things we can _____ . [Properties]

23. Methods all have _____ at the end of them. [Methods]

24. Having classes _____ objects helps ensure consistency among objects. [Instantiation]

25. A date variable with no date specified will return the _____ date. [Date Object and Time]

26. JavaScript uses numbers _____ through _____ as month numbers. [Retrieving Date Parts]

27. The code needed to convert a date into US format is _____ . [Localization]

28. When we subtract two dates in JavaScript, the difference appears in _____ . [Add and Subtract Dates]

Built-In Math Functions

29. In JavaScript, a _____ is an escape character. [Random]

30. The round method in JavaScript rounds a number to the nearest _____ number. [Round]

31. The Math.abs method in JavaScript will make a number represent its _____ value. [Abs]

32. The floor method takes the lower whole number of two numbers around a _____ . [Floor and Ceiling]

33. The min and max functions retrieve minimum and maximum numbers from a _____ of numbers. [Min and Max]

34. The Math.pow method in JavaScript takes a number and _____ it to the power specified. [Pow]

35. The Math.sqrt method takes a number and calculates its _____ [Sqrt]

Parameters and Return Values

36. _____ help to create reusable code. [Reusable Code]

37. The best way to test variable scope is to use the _____ . [Local vs. Global Scope]

38. _____ are the type of variable that cannot be redefined. [Redefining Variables]

39. When declaring a variable for use as a parameter, the _____ for the variable must be specified. [Passing Parameters]

40. For objects, it is better to set each object to have its own _____ . [Value vs. Reference]

41. If a _____ result is saved for later use, the return keyword is used. [Return Values]

Number, Boolean, and String Variables

A variable is one of the most important components of any programming language. A variable is a container that stores information, with the presumption that the variable will be used one or more times within a block of code. For example, if one must use the same sales tax rate five times within code, which option is better: entering the number five times, or storing that number in a variable once and then using that variable five times? Most would say the latter.

There are two aspects of data types you need to know about for the exam. First, variables have some naming rules. They can't have spaces and they cannot start with a number. They can have underscores and dollar signs. Variable names are case sensitive as well. For example, a variable named `saletax` is different from one named `salesTax` (the preferred camel casing) and different from one named `SalesTax`. Also, JavaScript is a loosely typed language, meaning that one does not need to specify a data type (like a number or string) when declaring a variable. JavaScript will look at the data and assign a data type accordingly.

The three variables you will declare in this project are number variables, string variables, which hold text, and Boolean variables, which are set to either true or false.

Purpose:

After completing this project, you will know how to name variables properly and you will know how to declare number, Boolean, and string variables.

Steps for Completion:

1. For each variable listed, indicate whether the variable name is valid or invalid:
 - a. `checkOut`:
 - b. `check Out`:
 - c. `$check_Out`:
 - d. `2checkOut`:
 - e. `checkOut2`:
2. Open the **21a-cart-variables.htm** file from the Music2 Student folder.
3. Scroll down to the script tag near the end of the document. You will see three commented lines.
4. Replace the text in the first commented line with a variable declaration setting a variable named `discount` set to a value of **10**
5. Replace the text in the second comment line with a variable declaration setting a variable named `checkOutExists` set to a value of **false**
6. Replace the text in the third comment line with a variable declaration setting a variable named `checkOutPage` set to the text, **checkout.htm**
7. Save your changes.
8. Preview the webpage in a web browser.
9. Click the Update Cart button. The total amount should be 860.72.
10. Click the Checkout button. A new window should open.
11. Click the Close Checkout button. The window opened in the previous step should close.

Project Details

Project Files:

21a-cart-variables.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Primitive Data Types

Subtopic: Number; Boolean; String

Objectives Covered:

2 Variables, Data Types, and Functions

- 2.1 Declare and use variables of primitive data types
 - 2.1.a Number
 - 2.1.b Boolean
 - 2.1.c String

Null and Undefined Variables

Variables of all types (number, Boolean, string, dates, and others) can be set to null or be left undefined. Here is the difference between the two: setting a variable to null makes it empty. This, however, presupposes that a variable had a value in the first place. An undefined variable is one that is declared but never has a value assigned to it. It is important to be able to distinguish the two states of null and undefined for the exam and beyond.

In addition, you will sometimes want to declare variables using the `var` keyword. While this is not required, using `var` tells JavaScript that indeed you are declaring a variable and not a new object. The less thinking JavaScript must do, the faster apps run.

Purpose:

After completing this project, you will be able to set a variable to null and set a variable to be undefined.

Steps for Completion:

1. Open the **21d-cart-variables.htm** file from the Music2 Student folder in a web browser.
2. In the Mandolin field, click the + button to add one to the quantity.
3. Click the Clear button. Notice the quantity does not clear but changes to 0.
4. To start to fix the code, open the **21d-cart-variables.htm** file in your code editor.
5. Scroll down about 65 to 70 lines to find this comment: `<!--make the manqty field empty when the Clear button is clicked-->`
6. A few lines below the comment, for the button with the ID, `manqtyzero`, change the code so that the `manqty` field becomes empty and not 0 when the button is clicked.
7. Scroll down to the bottom of the document.
8. Within the script tag, replace the comment with two variable declarations: `subtotal` and `grandTotal`. Both should be declared as undefined variables.
9. Save your changes.
10. Preview the webpage in a web browser.
11. In the mandolin area, click the + button to increase the quantity by one.
12. Click the Clear button. The quantity should now be empty.

Project Details

Project File:

21d-cart-variables.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Primitive Data Types

Subtopics: Null; Undefined

Objectives Covered:

2 Variables, Data Types, and Functions

2.1 Declare and use variables of primitive data types

2.1.d Null

2.1.e Undefined

Type of Operators and Type Checking Functions

When working through JavaScript code, you sometimes will need to check to see what data type is assigned to a variable. This is necessary because JavaScript is a loosely typed language, meaning data types are not explicitly assigned to variables. Rather, the data type is assigned when the variable receives a value. The `typeof` function, whether used in code or in a web browser's console, indicates the data type of a variable.

Similar to the `typeof` function, there are several other functions that can check to see if a variable has the anticipated data type. As an example, the `Number.isInteger` function checks to see if a variable is an integer.

Purpose:

After completing this project, you will know how to use the `typeof` function and type checking functions in JavaScript.

Steps for Completion:

1. Open the **2If-cart-variables.htm** file from the Music2 Student folder.
2. Scroll down to the bottom of the page and locate the script tag. There are three variables you will use to check properties: `discount`, `checkoutExists`, and `checkoutPage`.
3. Preview the webpage in a web browser.
4. Press the **F12** key to bring up the console. If necessary, click the Console tab.
5. In the Console tab, use the `typeof` function to identify the type for the `checkoutExists` variable. Which type is it?
6. Which variable type is the `checkoutPage` variable?
7. Is the `discount` variable an integer?

Project Details

Project Files:

2If-cart-variables.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 2

Topic: Primitive Data Types

Subtopic: `typeof` Operator; Type Checking Functions

Objectives Covered:

2 Variables, Data Types, and Functions

2.1 Declare and use variables of primitive data types

2.1.f Type of operator

2.1.g Type checking functions

The Use Strict Phrase

In an earlier project, we mentioned that the `var` keyword in front of a variable declaration tells JavaScript that the variable is indeed a variable and not an object. Using the `var` keyword in front of a variable also makes it easier for someone reading code to tell a variable apart from an object.

To enforce the use of the `var` keyword, the phrase, `use strict`, can be added within a script or a function.

Purpose:

After completing this project, you will know how to use the phrase, `use strict`, to force all variables to be declared with the `var` keyword.

Steps for Completion:

1. Open the **21h-practice-strict.htm** file from your Music2 Student folder.
2. Locate the script tag and the `getPracticeHours` function at the bottom of the page.
3. Add a blank line just after the `getPracticeHours` function declaration.
4. Add the phrase, **`use strict`**, with appropriate punctuation, to the blank line.
5. Save your changes.
6. Preview the webpage in a web browser.
7. Enter a number in the hours field.
8. Press the Enter key. What happened?
9. Go back and look at the code in your code editor.
Why did the webpage exhibit the behavior it just did?
10. Fix the cause of the error.
11. Save your changes.
12. Preview the webpage in a web browser.
13. Enter a number in the hours field.
14. Press the Enter key. The webpage should now work.

Project Details

Project Files:

21h-practice-strict.htm

Estimated Completion Time:

15 minutes

Video Reference:

Domain 2

Topic: Primitive Data Types

Subtopic: Use Strict

Objectives Covered:

2 Variables, Data Types, and Functions

2.1 Declare and use variables of primitive data types

2.1.h Use strict

Converting Between Data Types

When input fields are created on an HTML form, they are objects in the eyes of JavaScript. Many of these fields eventually contain numbers used in a calculation. The only problem with this is that because these are objects, they cannot be used in calculations unless they are converted to a number, like an integer or a floating decimal. This is just one of many examples of data type conversions that can become necessary on occasion in JavaScript.

Purpose:

After completing this project, you will have a better understanding of when data type conversions are needed and how to perform those conversions.

Steps for Completion:

1. Open the **2li-cart-operators.htm** file from the Music2 Student folder in a web browser.
2. Press the **F12** key to bring up the console.
3. In the console, type: **typeof(document.getElementById('mantotal'))**
4. Press the Enter key. Is this an object or a variable?
5. Open the **2li-cart-operators.htm** file in your code editor.
6. Scroll down until you see this comment: `<!-- replace comments with the command necessary to make these values decimals -->`
7. Within the orderbuttons div class, replace each comment with the built-in function used to convert an object into a decimal field.
8. Save your changes.
9. Preview the webpage in a web browser.
10. Click the Update Cart button. Do not worry about the floating decimal as you will fix that in the next project.

Project Details

Project Files:

2li-cart-operators.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Primitive Data Types

Subtopic: Convert Between Data Types

Objectives Covered:

2 Variables, Data Types, and Functions

2.1 Declare and use variables of primitive data types

2.1.i Converting between data types (parseInt, parseFloat)

Formatting Numbers

One of the most common needs for displaying data in JavaScript is the need to format numbers, especially those with a float data type. Floating numbers tend to display more places after the decimal point than desired. The `toFixed` method sets the number of places to display after the decimal point. This is just one of many situations in which numbers can be formatted.

Purpose:

After completing this project, you will be able to format numbers to improve their look on a webpage.

Steps for Completion:

1. Open the **21j-formatting-numbers.htm** file from the Music2 Student folder in a web browser.
2. Click the Update Cart button. The sales tax and total amounts should be formatted to two decimal places, not the number showing.
3. Open the **21j-formatting-numbers.htm** file in your code editor.
4. Scroll down near the end of the file, specifically to this comment: `<!-- the sales tax and the total need to be converted to two decimal places -->`
5. Inside the button code inside of the `order_buttons` div class, format the `salesTax` element to show two numbers after the decimal point.
6. Format the `grandtotal` element to show two numbers after the decimal point.
7. Save your changes.
8. Preview the webpage in a web browser.
9. Click the Update Cart button. The sales tax and total should now display with two decimal places.

Project Details

Project File:

21j-formatting-numbers.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Primitive Data Types

Subtopic: Format Numbers

Objectives Covered:

2 Variables, Data Types, and Functions

2.1 Declare and use variables of primitive data types

2.1.j Formatting numbers

String Operations

There are many ways to manipulate strings. To concatenate strings is to combine pieces of strings together. Knowing the length of a string will determine if it will fit a defined space in a document. String operations can also include controlling how numbers display within a webpage.

Another string operation to consider is that of nesting quotes. Single quotes can be nested within double quotes and vice versa. The key is knowing which quotes you wish to display, if applicable. Those should be the inner quotes within a set of nested quotes.

Purpose:

Upon completing this project, you will know how to find the length of a string and be able to concatenate strings. You will also know how to control nested quotes.

Steps for Completion:

1. Open the **21k-practice-strings.htm** file from the Music2 folder in a web browser.
2. Adjust msg2 to display as a number set to three digits.
3. Save your changes.
4. Preview the webpage in a web browser.
5. Enter **40** for the number of hours you can practice and select the Enter key.
6. Open the Console.
7. Use the Console to find the length of the practiceweeks paragraph string. What is the length of the practiceweeks paragraph string?
8. Return to your code editor.
9. Change the quote structure in msg7 to have the phrase, dream?, in single quotes.
10. Save your changes.
11. Preview the webpage in a web browser.
12. Enter **40** for the number of hours you can practice and select the Enter key. The phrase, dream?, should be in single quotes.

Project Details

Project Files:

21k-practice-strings.htm

Estimated Completion Time:

15 minutes

Video Reference:

Domain 2

Topic: Primitive Data Types

Subtopic: String Operations;
Nested Quotes

Objectives Covered:

2 Variables, Data Types, and Functions

2.1 Declare and use variables of primitive data types

2.1.k String operations

2.1.l Single quote vs. double quote (nesting)

LearnKey Initialization

Initialization is storing or assigning a value in a variable. A declaration is the creation of a variable without it being initialized. A variable can be initialized at any time.

Purpose:

Upon completing this project, you will know the difference between declaration and initialization.

Steps for Completion:

1. Open the **21m-cart-variables.htm** file from the Music2 Student folder in a web browser.
2. In the Script section, which of the five declared variables have been initialized?
3. Which of the variables have been declared but not initialized?
4. Close all open files in both your web browser and your code editor.

Project Details

Project Files:

21m-cart-variables.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 2

Topic: Primitive Data Types

Subtopic: Initialization

Objectives Covered:

2 Variables, Data Types, and Functions

2.1 Declare and use variables of primitive data types

2.1.m Initialization

Build and Iterate Through Arrays

An array has multiple containers that contain multiple values. Where a variable has one container with one value, an array will have multiple containers, each storing an item. A single-dimensional array has one row of containers within its box and these containers will have a single value in each. The single-dimensional array will show all the items in the array on one line within one set of brackets. Similar to variables, arrays can be declared and initialized at the same time or declared and then initialized later.

The array length is needed when iterating through an array. A loop is used to iterate through an array and will connect all the containers of an array and allow easy repetition of an action while the conditions are being met. This will allow you to change the style of a single-dimensional array.

Purpose:

Upon completing this project, you will be able to understand and build the code of a single-dimensional array. You will also be able to use a loop to iterate through an array.

Steps for Completion:

1. Open the **22a-tuningkits-array.htm** file from the Music2 Student folder.
2. Scroll down to the comment, `// declare array here`.
3. Replace the comment with a single-dimensional array named **tuningList**
4. Within the tuningList array, add the values, **Tuning Lever** and **Tuning Fork**
5. Save your changes.
6. Preview the webpage in a web browser.
7. Click the Update List button. You should see the page update with the array values and a message that says, **A tuning lever and tuning fork is a must-have for any tuning kit**
8. Return to the code editor.
9. Uncomment the lines containing the opening and closing ul tags for the tuningList HTML element.
10. Replace the text, `// insert loop here`, with a for loop that iterates through the elements in the tuningList array, ensuring that each element appears as a bullet.
11. Save your changes.
12. Preview the webpage in a web browser.
13. Click the Update List button. The array values should now appear as a bulleted list.

Project Details

Project Files:

22a-tuningkits-array.htm

Estimated Completion Time:

15 minutes

Video Reference:

Domain 2

Topic: Declare and Use Arrays

Subtopic: Build and Define an Array; Iteration and Length; Initialization

Objectives Covered:

2 Variables, Data Types, and Functions

2.2 Declare and use arrays

2.2.a Single-dimensional arrays

2.2.c Iteration

2.2.d Initialization

2.2.e Define, sort, and search an array

Sorting and Searching

Sorting and searching an array will allow you to find content that should or should not be in the array. Sorting an array list will display the list in alphabetical order. When searching for an array element, understand that the first element in an array is element 0. Often, one will use the Console to search for an array element. If an element is in an array, it will return its position. If not, it will return a -1.

Purpose:

Upon completing this project, you will be able to sort an array and search for content in an array.

Steps for Completion:

1. Open the **22e-tuningkits-array.htm** file from the Music2 Student folder.
2. On the first line inside of the `updateList` function, add code to sort the `tuningList` array in alphabetical order.
3. Save your changes.
4. Preview the webpage in a web browser.
5. Click the Update List button. In the bulleted list that appears, the tuning fork should appear before the tuning lever.
6. Open the Console.
7. Search the array in the Console to find out if the Tuning Lever has a number.
8. Does Tuning Lever have a number in the array?
9. Search the array to find out if Felt Strips has a number.
10. Does Felt Strips have a number in the array?

Project Details

Project Files:

22e-tuningkits-array.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Declare and Use Arrays

Subtopic: Sorting and Searching

Objectives Covered:

2 Variables, Data Types, and Functions

2.2 Declare and use arrays

2.2.e Define, sort, and search an array

Push and Pop

JavaScript has commands we can use to control what is in an array after the array has been initialized. Push will add an item to an array, while pop will remove the last item in an array. Pop is best used in a last in, first out situation.

Purpose:

Upon completing this project, you will be able to push an item into an array, and pop the last item out of the array.

Steps for Completion:

1. Open the **22f-tuningkits-array.htm** file from the Music2 Student folder.
2. Find the first article with the class, `threecol1`.
3. For the input check box with the ID of `electronicTuner`, add an onclick event with a function that will push the item, `Electronic tuner`, to the `tuningList` array.
4. For the input check box with the ID of `wedgeMute`, add an onclick event with a function that will push the item, `Wedge mute` to the `tuningList` array.
5. For the input check box with the ID of `pappsMute`, add an onclick event with a function that will push the item, `Papps mute`, to the `tuningList` array.
6. For the input check box with the ID of `feltStrips`, add an onclick event with a function that will push the item, `Felt strips`, to the `tuningList` array.
7. Save your changes.
8. Preview the webpage in a web browser.
9. Select the **Wedge mute** check box.
10. Click **Update List**. The tuning list should show three bulleted items.
11. Open the Console.
12. In the Console, pop a list item from the array. Which item was removed from the list?
13. Check that the `Wedge mute` item is no longer in the array.

Project Details

Project Files:

22f-tuningkits-array.htm

Estimated Completion

Time:

15 minutes

Video Reference:

Domain 2

Topic: Declare and Use Arrays

Subtopic: Push and Pop

Objectives Covered:

2 Variables, Data Types, and Functions

2.2 Declare and use arrays

2.2.f Push, pop, shift, and unshift methods

Shift and Unshift

Shift will remove the first item in an array, while unshift will add an item to the beginning of an array. Shift is similar to pop as they both remove items from an array, though pop removes an item from the end of an array. Unshift is similar to push as both commands add to an array, though push adds an item to the end of an array.

Purpose:

Upon completing this project, you will be able to know when to use the Console or array code to add or remove items from an array. You will be able to use shift to remove items from an array and unshift to add items to an array.

Steps for Completion:

1. Open the **22g-tuningkits-array.htm** file from your Music2 Student folder in a browser.
2. Open the Console.
3. Shift a list item from the **tuningList** array. What list item was removed from the array?
4. Open the **22g-tuningkits-array.htm** file in your code editor.
5. Within each of the input check box tags for the threecoll articles, change the onClick functions to run from push to unshift.
6. Within the updateList function, comment out the tuningList.sort method.
7. Save your changes.
8. Preview the webpage in a web browser.
9. Select the Wedge mute check box.
10. Click the Update List button. Where did the unshift place Wedge mute in the tuningList array?

Project Details

Project Files:

22g-tuningkits-array.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Declare and Use Arrays

Subtopic: Shift and Unshift

Objectives Covered:

2 Variables, Data Types, and Functions

2.2 Declare and use arrays

2.2.f Push, pop, shift, and unshift methods

Access an Element

The array index number is used to access an element in an array. The first item in an array is index number zero. After that, each item number increments by one through the arrays.

Purpose:

Upon completing this project, you will be able to access an array element using an index number.

Steps for Completion:

1. Open the **22h-tuningkits-array.htm** file from the Music2 Student folder.
2. At the end of the `updateList` function, create an alert that says **The first item in this array is** and use the index number for the first item in the array to access the element.
3. Save your changes.
4. Preview the webpage in a web browser.
5. Select the Wedge mute check box.
6. Click the Update List button. You should get an alert indicating that the first item in the array is the Wedge mute.

Project Details

Project Files:

22h-tuningkits-array.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 2

Topic: Declare and Use Arrays

Subtopic: Access an Element;
Understanding Multi-Dimensional
Arrays

Objectives Covered:

2 Variables, Data Types, and
Functions

2.2 Declare and use arrays

2.2.g Length property

2.2.h Access an array element

Using Multi-Dimensional Arrays

To this point, arrays we have seen have been single-dimensional arrays, that is, array lists with one group of values. Arrays, however, can be multi-dimensional, as they can contain a group and then within each group, another group. Here is a real-life example: an array contains a list of locations, and within each location, amenities. The location and the amenities would serve as the two dimensions.

Purpose:

Upon completing this project, you will better understand and be able to use multi-dimensional arrays.

Steps for Completion:

1. Open the **22j-tuningkits-array.htm** file from the Music2 Student folder.
2. Notice the multiArray function, which declares a two-dimensional array with two elements in each container.
3. Replace the comment, // set the 0,0 element of aI, with the code that will set the 0,0 element of aI to Tuning Fork and Red.
4. Save your changes.
5. Preview the webpage in a web browser. You should see the array at the bottom of the page.
6. Close all of your open files in both your browser and your code editor.

Project Details

Project Files:

22j-tuningkits-array.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 2

Topic: Declare and Use Arrays

Subtopic: Access an Element;
Understanding Multi-Dimensional
Arrays

Objectives Covered:

2 Variables, Data Types, and
Functions

2.2 Declare and use arrays

2.2.b Multi-dimensional
arrays

Properties and Methods

A property is a characteristic of an object and a method is an action toward an object or property. A method will always have parentheses at the end to specify what is being taken action on.

Purpose:

Upon completing this project, you will be able to recognize both properties and methods and write code using methods.

Steps for Completion:

1. For each expression, identify whether the expression is a property or method:
 - a. `innerHTML`:
 - b. `document.getElementById`:
 - c. `push`:
 - d. `length`:
2. Open the **23a-cart-constants.htm** file from the Music2 Student folder.
3. Find the comment, `// set the discount to two decimal points`.
4. On the line below the comment, add a method to the discount result that will set the result to be fixed to two decimal points.
5. Save your changes.
6. Preview the webpage in a web browser. The dollar amounts at the bottom of the page should be set to two decimal points.

Project Details

Project Files:

23a-cart-constants.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 2

Topic: Complete and Debug Code on Objects

Subtopic: Properties; Methods

Objectives Covered:

2 Variables, Data Types, and Functions

2.3 Complete and debug code that uses objects

2.3.a Properties

2.3.b Methods

Instantiation

Instantiation is the creation of a new template or class for an object. When a class is used to define an object, it creates consistency among objects and the manner in which they are created. For this reason, classes are often created in an external JavaScript file, so that multiple files can access the class.

Purpose:

Upon completing this project, you will be able to instantiate a class in JavaScript to create consistency.

Steps for Completion:

1. Open the **23c-main.js** file from the Music2 Student scripts folder.
2. Replace the comment, // insert product class here, with a class for a new product that will contain three objects: item, size, and price.
3. Save your changes.
4. Open the **23c-products-class.htm** file from the Music2 Student folder.
5. Below the footer, add an opening and closing script tag.
6. Within the script tag, add a new instance of the product class named **mandolin** with the following details:
 - a. item: **Mandolin**
 - b. size: **36"**
 - c. price: **379.99**
7. Within the script tag, add a new instance of the product class named **drum** with the following details:
 - a. item: **Drum**
 - b. size: **32"**
 - c. price: **129.99**
8. Save your changes.
9. Preview the webpage in a web browser.
10. Open the Console.
11. Return the size of the mandolin. It should be 36".
12. Return the price of the drum. It should be 129.99.

Project Details

Project Files:

23c-products-class.htm
23c-main.js

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain 2

Topic: Complete and Debug Code on Objects

Subtopic: Instantiation

Objectives Covered:

2 Variables, Data Types, and Functions

2.3 Complete and debug code that uses objects

2.3.c Instantiation

Date Object and Time

The date and time object can be used to build a date variable and retrieve date parts using mathematics and built-in JavaScript methods.

Purpose:

Upon completing this project, you will be able to build date variables and retrieve date parts.

Steps for Completion:

1. Open the **23d-practice-dates.htm** file from the Music2 Student folder.
2. Just below the opening script tag, declare a date variable named **lessonStartDate** with a fixed date and time of **October 2, 2018 at 6:00 PM**.
3. On the next line, declare a date variable named **today** with the current date.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Open a Console window.
7. Display the lessonStartDate variable. It should show a date of October 2, 2018.
8. Display the today variable. It should show the current date.

Project Details

Project Files:

23d-practice-dates.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Complete and Debug Code on Objects

Subtopic: Date Object and Time

Objectives Covered:

2 Variables, Data Types, and Functions

2.3 Complete and debug code that uses objects

2.3.d Date object

Retrieving Date Parts

JavaScript will return most date parts as numbers (including for months, which are numbered 0 through 11). This complicates returning a month name. The month, day, and year of a date must be retrieved individually, using the `getMonth`, `getDay`, and `getFullYear` methods, respectively. To return the month name you will need to build an array with all the month names and tie the array to the month number.

Purpose:

Upon completing this project, you will be able to retrieve date parts and have the month name returned as the month value.

Steps for Completion:

1. Open the **23e-practice-dates.htm** file from the Music2 Student folder.
2. Replace the comment, `// get the month of the lesson`, with the month of the `lessonStartDate` variable.
3. Save your changes.
4. Preview the webpage in a web browser.
5. Click in the Enter the number of hours field.
6. Type: **20** and press the Enter key. Notice the message that displays that classes start in and then a month number.
7. Return to your code editor.
8. Replace the comment, `// declare month array here`, with a variable named `monthArray` that has an array of all 12 months of the year.
9. On the line where you obtained the month of the `lessonStartDate` variable, change the method to get the month name from the `monthName` array.
10. Save your changes.
11. Preview the webpage in a web browser.
12. Click in the Enter the number of hours field.
13. Type: **20** and press the Enter key. Notice the message that displays that classes start in and then a month instead of a month number.

Project Details

Project Files:

23e-practice-dates.htm

Estimated Completion Time:

15 minutes

Video Reference:

Domain 2

Topic: Complete and Debug Code on Objects

Subtopic: Retrieving Date Parts

Objectives Covered:

2 Variables, Data Types, and Functions

2.3 Complete and debug code that uses objects

2.3.e Retrieve date and time parts

Localization

Date localization uses the `toLocaleDateString` method to have the date display in a localized format. As an example, location date formats are `mm/dd/yyyy` for the USA and `dd/mm/yyyy` for the UK.

Purpose:

Upon completing this project, you will be able to localize dates using JavaScript.

Steps for Completion:

1. Open the **23f-practice-dates.htm** file from the Music2 Student folder.
2. Replace the comment, `// insert localized date`, with a reference to the `lessonStartDate` variable, formatted in the UK format.
3. Save your changes.
4. Preview the webpage in a web browser.
5. Click in the Enter the number of hours field.
6. Type: **20** and press the Enter key. Notice that the start date is in the UK format of `dd/mm/yyyy`.
7. Return to your code editor.
8. Change the locale of the start date to display in `mm/dd/yyyy` format.
9. Save your changes.
10. Preview the webpage in a web browser.
11. Click in the Enter the number of hours field.
12. Type: **20** and press the Enter key. Notice that the start date is in the US format of `mm/dd/yyyy`.

Project Details

Project Files:

23f-practice-dates.htm

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain 2

Topic: Complete and Debug Code on Objects

Subtopic: Localization

Objectives Covered:

2 Variables, Data Types, and Functions

2.3 Complete and debug code that uses objects

2.3.f Localize date format (MM/DD vs DD/MM)

Add and Subtract Dates

Adding and subtracting dates in JavaScript use different default intervals. When a number is added to a date, JavaScript assumes the number to be the number of days to add. When two dates are subtracted in JavaScript, the return will be in milliseconds. The code will need to be written so that 86400000 milliseconds in a day will be divided into the returned milliseconds from the subtraction to return the number of days.

Purpose:

Upon completing this project, you will be able to add and subtract dates using JavaScript.

Steps for Completion:

1. Open the **23g-practice-dates.htm** file from the Music2 Student folder.
2. Replace the comment, `// declare lessonEndDate here` with a variable named `lessonEndDate` that is set to the `lessonStartDate`.
3. On the next line, set the `lessonEndDate` to be 56 days after the `lessonStartDate`.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Click in the Enter the number of hours field.
7. Type: **20** and press the Enter key. Notice the end date of 11/27/2018.
8. Open a Console.
9. Calculate the difference between the `lessonEndDate` and the `lessonStartDate`. What is the difference in milliseconds?
10. Change the calculation so that the result is returned in weeks. You should see 8 weeks as a result.
11. Close your open files, both in the browser and in your code editor.

Project Details

Project Files:

23g-practice-dates.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Complete and Debug Code on Objects

Subtopic: Add and Subtract Dates

Objectives Covered:

2 Variables, Data Types, and Functions

2.3 Complete and debug code that uses objects

2.3.g Add and subtract dates

Random Numbers

A random number generator can be used when creating an online quiz or game with JavaScript. The `Math.random` method will return the numbers 0 or 1. The number returned should then be multiplied by the number of random choices needed for the project and set to use the `Math.floor` function if you want the lowest possible integer returned (0 and 1 for two numbers, for example).

Purpose:

Upon completing this project, you will be able to use the `Math.random` method to allow JavaScript to choose randomly between two videos to display upon webpage loading.

Steps for Completion:

1. Open the **24a-jukebox.htm** file from the Music2 Student folder.
2. Between the opening and closing script tags at the bottom of the page, add a function named **loadJukebox**
3. Within the `loadJukebox` function, add a variable named **videoNumber** that is set to generate a random number of either 0 or 1.
4. Add an if else statement that satisfies the following conditions:
 - a. If the `videoNumber` is 0, set the source of the HTML element `videoJukebox` to the `Blues.mp4` video in the Videos folder.
 - b. If the `videoNumber` is 1, set the source of the HTML element `videoJukebox` to the `Happy.mp4` video in the Videos folder.
5. Save your changes.
6. Preview the webpage in a web browser.
7. Refresh the webpage a few times. Sometimes a happy song will be ready to play and sometimes a blues song will be ready to play.

Project Details

Project Files:

24a-jukebox.htm
Blues.mp4
Happy.mp4

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 2

Topic: Built-In Math Functions

Subtopic: Random

Objectives Covered:

2 Variables, Data Types, and Functions

2.4 Complete and debug code that uses built-in Math functions

2.4.a Random

Rounding and Absolute Methods

The round method will round a number to the nearest whole number. The Math.round method in JavaScript is used to round numbers. The absolute value of a number, represented by the Math.abs method, is the absolute distance of a number from zero. For example, -3 and 3 both have an absolute value of 3.

Purpose:

Upon completing this project, you will be able to write code using the Math.round method to round a number to the nearest whole number using JavaScript. You will also be able to use the Math.abs method to get the absolute value of a number.

Steps for Completion:

1. Open the **24b-practice-numbers.htm** file from the Music2 Student folder.
2. Within the getPracticeHours function, round the result for the msg2 variable to the nearest whole number.
3. Within the getPracticeHours function, make the hoursEntered variable calculation an absolute value.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Click in the Enter the number of hours field.
7. Type: **-20** and press the Enter key. You should see the result round up to one week and the number one should be a positive integer since the Math.abs method was used.

Project Details

Project Files:

24b-practice-numbers.htm

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 2

Topic: Built-In Math Functions

Subtopic: Round; Abs

Objectives Covered:

2 Variables, Data Types, and Functions

2.4 Complete and debug code that uses built-in Math functions

2.4.b Round

2.4.c Abs

Floor and Ceiling

The floor and ceiling method will not take into consideration what is after the decimal point in a number when displaying the number as an integer. To return the nearest lower whole number, use the `Math.floor` method. To return the nearest higher whole number, use the `Math.ceil` method. At times, the floor or ceiling method can return a more precise whole number than the round method. When calculating days or time, the highest whole number is often needed, thus making it necessary to use the `Math.ceil` method.

Purpose:

Upon completing this project, you will be able to determine which method is best to use between the `Math.floor` and `Math.ceil` methods to get a more precise whole number return.

Steps for Completion:

1. Open the **24d-practice-numbers.htm** file from the Music2 folder in a web browser.
2. Click in the Enter the number of hours field.
3. Type: **6** and press the Enter key. Notice that the results show 3 hours a week of practice, which, multiplied by 6 weeks, falls short of the total of 20 hours.
4. Open the **24d-practice-numbers.htm** file in your code editor.
5. For the `msg2` variable, replace the `Math.round` method with a method that will always move the variable up to the next integer.
6. Save your changes.
7. Preview the webpage in a web browser.
8. Click in the Enter the number of hours field.
9. Type: **6** and press the Enter key. Notice that the results now show 4 hours a week of practice, which, multiplied by 6 weeks, exceeds the total of 20 hours.
10. Open a Console.
11. Type: **`Math.floor (20/6)`** and press the Enter key. What is the result?

Project Details

Project Files:

24d-practice-numbers.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Built-In Math Functions

Subtopic: Floor and Ceiling

Objectives Covered:

2 Variables, Data Types, and Functions

2.4 Complete and debug code that uses built-in Math functions

2.4.d Floor

2.4.e Ceil

Min and Max

Min and max will retrieve the minimum or maximum number in a range of numbers, respectively. Variables are usually used to create the range to pull the data from using the Math.min or Math.max method.

Purpose:

Upon completing this project, you will be able to create variables that will return the min and max prices from a list of prices.

Steps for Completion:

1. Open the **24f-cart-max-min.htm** file from the Music2 Student folder.
2. Near the end of the webpage, replace the comment starting with the phrase, get the minimum, with a method that will get the lowest price among these three variables: manprice, drumprice, and tromprice.
3. Replace the comment starting with the phrase, get the maximum, with a method that will get the highest price among these three variables: manprice, drumprice, and tromprice.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Click the Update Cart button. You should see the lowest and highest priced items.

Project Details

Project Files:

24f-cart-max-min.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 2

Topic: Built-In Math Functions

Subtopic: Min and Max

Objectives Covered:

2 Variables, Data Types, and Functions

2.4 Complete and debug code that uses built-in Math functions

2.4.f Min

2.4.g Max

Powers and Square Roots

The `Math.pow` method will take a number and raise it according to the power number specified. When a number is raised to the power of a number, it takes the number multiplied by the same number as many times as the power specifies.

The `Math.sqrt` method will take a number and calculate the square root of the number. As an example, this is used to determine the length of a side if the square footage is known.

Purpose:

Upon completing this project, you will be able to write JavaScript code that will use the `Math.pow` method to find the square footage for the user of a webpage. You will also be able to use the `Math.sqrt` method to calculate and return the square root from the input of the square footage.

Steps for Completion:

1. Open the **24g-square-footage.htm** file from the Music2 folder.
2. In the `getSquareFootage` function, replace the comment, `// add power method`, with the method use to raise the integer being parsed to the second power.
3. Replace the comment, `// add square root method`, with the method used to obtain the square root of the integer being parsed.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Enter the number **441** in the last text box on the webpage.
7. Click the Square Footage button. What is the result?
8. Click the Side button. What is the result?
9. Close all of your open files in both your browser and your code editor.

Project Details

Project Files:

24g-square-footage.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Built-In Math Functions

Subtopic: Pow; Sqrt

Objectives Covered:

2 Variables, Data Types, and Functions

2.4 Complete and debug code that uses built-in Math functions

2.4.h Pow

2.4.i Sqrt

Reusable Code

Functions are created so there is reusable code that can be used in multiple areas. If changes need to be made to code, with a function, the code will only need to be changed in one place instead of multiple places. When code is reused it also lessens the amount of code to write and organizes the code.

Purpose:

Upon completing this project, you will be able to create functions that allow you to reuse code and organize the code.

Steps for Completion:

1. Open the **25a-cart-numbers.htm** file from the Music2 Student folder.
2. Replace the comment, // add addOne function, with a function named addOne that will do the following:
 - a. Store the existing quantity of a field in a variable named qty.
 - b. Use a variable named newQty to parse, from the HTML element with the qty field, the value into an integer.
 - c. Increment the newQty variable by one.
 - d. Assign the newQty value to the HTML element with the qty field.
3. Find the comment, // add reference to addOne function using manqty.
4. Replace the comment with a call to the addOne function using the manqty element.
5. Save your changes.
6. Preview the webpage in a web browser.
7. Click the + button on the mandolin item. The quantity should increase by one.

Project Details

Project Files:

25a-cart-numbers.htm

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 2

Topic: Parameters and Return Values

Subtopic: Reusable Code

Objectives Covered:

2 Variables, Data Types, and Functions

2.5 Complete and debug functions that accept parameters and return values

2.5.a Reusable code

Scopes and Variable Definitions

A scope describes the depth, or availability of use, that a variable can be used in a file. There are two factors that determine the scope: where the variable is declared, and if it is specifically being declared by using the var keyword. Local variables are only available within their function and have a local scope. Global variables are available anytime and have a global scope. A variable declared outside of functions or that does not use the var keyword has a global scope.

Purpose:

Upon completing this project, you will know the difference between local and global variables and where they are available to use. You will also be able to identify parameters within functions.

Steps for Completion:

1. Open the **25b-cart-scopes.htm** file from the Music2 Student folder.
2. Open the **25b-main.js** file from the Music2 Student scripts folder.
3. In the main.js file, is the incMsg variable local or global?
4. Which variable cannot be redefined?
5. Switch to the 25b-cart-scopes.htm file.
6. Scroll down to the script section near the end of the document. Which variables are redefined within their functions?
7. Is the qty variable a local or global variable?
8. Within the script, which functions are using parameters?

Project Details

Project Files:

25b-cart-scopes.htm
25b-main.js

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 2

Topic: Parameters and Return Values

Subtopic: Local vs. Global Scope;
Redefining Variables; Passing
Parameters

Objectives Covered:

2 Variables, Data Types, and
Functions

2.5 Complete and debug functions
that accept parameters and return
values

2.5.b Local vs. global scope

2.5.c Redefine variables

2.5.d Pass parameters

Value vs. Reference

Primitive data types can be easily set to be equal to another primitive data type, while objects cannot be changed in this way. An object references shared data space, so every object sharing the space will change. Primitive data types will return a value that passes method arguments and creates a return. Object variables are referenced to a shared space and do not pass method arguments. With objects, it is best practice to set each object with its own value.

Purpose:

Upon completing this project, you will know the difference between a value and a reference.

Steps for Completion:

1. Open the **25e-cart-numbers.htm** file from the Music2 Student folder in a web browser.
2. Open the Console.
3. In the Console, declare a variable of **x** and set it to **4**
4. Declare a variable of **y** and set it to **x**.
5. Set **y** to be equal to **7**
6. What is the value of **x** after **y** has been made equal to **7**?
7. If **x** and **y** were an array, would the value of **x** stay the same?
8. Why will the object value change when it is set equal to another object?
9. Can an object have a set value?
10. What needs to be done for an object to have a set value?

Project Details

Project Files:

25e-cart-numbers.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Parameters and Return Values

Subtopic: Value vs. Reference

Objectives Covered:

2 Variables, Data Types, and Functions

2.5 Complete and debug functions that accept parameters and return values

2.5.e Value vs. reference

Return Values

A return value is an answer to a function calculation. The return keyword is used to return a function result that is then used in another process, such as assigning the result to an HTML element, for example.

Purpose:

Upon completing this project, you will be able to use a return keyword as part of a function.

Steps for Completion:

1. Open the **25f-practice-numbers.htm** file from the Music2 Student folder.
2. In the lessonSchedule function near the bottom of the page, create a return statement that will return the month by name (Hint: look for code to help generate a month name).
3. In the body tag, set the lessonSchedule function to run when the webpage loads.
4. Save your changes.
5. Preview the webpage in a web browser. You should see an alert with the current month.
6. Close all of your open files in your code editor and your browser.

Project Details

Project Files:

25f-practice-numbers.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 2

Topic: Parameters and Return Values

Subtopic: Return Values

Objectives Covered:

2 Variables, Data Types, and Functions

2.5 Complete and debug functions that accept parameters and return values

2.5.f Return values

JavaScript

Domain 3

Presented By
LK LearnKey®

Fill-in-the-Blanks Domain 3

Instructions: While watching Domain 3, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

Logical and Comparison Operators

1. Evaluation expressions compare two parts, usually an _____ and a _____, and return a true or false value. [Equals]
2. In JavaScript, the symbol representing that something is not equal to a value is the _____ symbol. [Not Equals]
3. The less than and greater than symbols in JavaScript are the same as they are for basic _____ elements. [Greater Than and Less Than]
4. A common logical problem in JavaScript is not to have the proper _____ for a less than or less than or equal to situation. [Less Than or Equal To]
5. With greater than and greater than or equal to, you must figure out if the _____ requires the number in question to be included. [Greater Than or Equal To]
6. We saw that the `!=` means not equal to, the `!` part is the _____ part and it can be used other places besides before an `=`. [Not]
7. The `and` symbol is two _____ . [And]
8. An `or` operator is a _____ symbol. [Or]

Decision Statements

9. In JavaScript, the condition for an `if` statement is always inside _____ . [If]
10. When using the `if` keyword to see if a situation is true, you may want to check for another situation in case the first situation is _____ . [Else If]
11. Unlike an _____ statement, a `switch` statement will not stop checking a condition after a first match. [Switch]
12. A nested `if` statement is an _____ statement inside of an `if` statement. [Nested If]

Loops

13. The `for` loop runs a set number of times, either set by us or set by a _____ . [For]
14. The `while` loop is defined as this: while this condition is _____ run this code. [While]
15. A `do` loop runs _____ and then checks the condition. [Do]
16. Sometimes within a _____ , we need to break code. [Break]
17. The `continue` keyword _____ one iteration of the loop based on a condition. [Continue]

Equals and Not Equals

The equal sign will set the variable equal to a value, while the double equal sign asks the following question: is the variable equal to a set value? The double equal is normally used in a statement such as an if statement, which is covered later in this domain. The sign used in JavaScript to represent a not equal is an exclamation point in front of an equal sign (!=).

Purpose:

Upon completing this project, you will be able to determine when to use the equal sign, the double equal sign, and the not equal sign.

Steps for Completion:

1. Open the **3 la-cart-evaluation.htm** file from the Music3 Student folder.
2. In the script section near the bottom of the page, replace the comment, // add set operator, with the correct operator to set the checkoutExists variable to false.
3. Replace the comment, // add conditional operator, with the correct operator to see if the checkoutExists variable is false.
4. Replace the comment, // add not equal operator, with the correct operator to see if the checkoutExists variable is not false.
5. Save your changes.
6. Preview the webpage in a web browser.
7. Click the Checkout button. A checkout window should appear.
8. Click the Close Checkout button. The checkout window should close.

Project Details

Project Files:

3 la-cart-evaluation.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 3

Topic: Logical and Comparison Operators

Subtopic: Equals; Not Equals

Objectives Covered:

3 Decisions and Loops

3.1 Evaluate expressions that use logical and comparison operators

3.1.a !=

3.1.g ==

Greater Than or Less Than

Greater than and less than comparison operators are normally used in conditional statements. A conditional statement is an if statement that returns a true or false.

Purpose:

Upon completing this project, you will be able to create a greater than statement and a less than statement that will return a specific message when a condition is met.

Steps for Completion:

1. Open the **3Ic-practice-comparison.htm** file from the Music3 Student folder.
2. Replace the comment, // are less than 3, with the correct operator to test if the hoursEntered are less than 3 hours.
3. Replace the comment, // are more than 8, with the correct operator to test if the hoursEntered are more than 8 hours.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Click in the Enter number of hours field.
7. Type: **2** and press the Enter key. You should see a message suggesting practicing at least three hours a week.
8. Type: **10** and press the Enter key. You should see a message suggesting that may be too much practice for a beginner.

Project Details

Project Files:

3Ic-practice-comparison.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 3

Topic: Logical and Comparison Operators

Subtopic: Greater Than and Less Than

Objectives Covered:

3 Decisions and Loops

3.I Evaluate expressions that use logical and comparison operators

3.I.b <

3.I.c >

Other Comparison Operators

The less than or equal to is a logical operator that will allow a number and anything less than the number to return a value. The greater than or equal to is a logical operator that will allow the number and anything greater than the number to be included in the returned value. It is important to understand the logic behind what you are creating to know when to use less than or equal to or just the less than sign as well as the greater than or equal to or greater than sign. You also want to make sure any messages to users convey your logic properly.

Purpose:

Upon completing this project, you will be able to use the less than or equal to sign and greater than or equal to sign to return a specific message when the operating conditions are met.

Steps for Completion:

1. Open the **31d-practice-comparison.htm** file from the Music3 Student folder.
2. Find the `hoursEntered < 3` condition and change it to include the number 3.
3. Change the accompanying message for the condition to indicate that more than three hours of practice are needed.
4. Find the `hoursEntered > 8` condition and change it to include the number 8.
5. Change the accompanying message for the condition to indicate that eight or more hours of practice may be too much.
6. Save your changes.
7. Preview the webpage in a web browser.
8. Click in the Enter the number of hours field.
9. Type: **3** and press the Enter key. You should see a message suggesting practicing more than three hours a week.
10. Type: **8** and press the Enter key. You should see a message suggesting that may be too much practice for a beginner.

Project Details

Project Files:

31d-practice-comparison.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 3

Topic: Logical and Comparison Operators

Subtopic: Less Than or Equal To; Greater Than or Equal To

Objectives Covered:

3 Decisions and Loops

3.1 Evaluate expressions that use logical and comparison operators

3.1.d <=

3.1.e >=

Not

An exclamation point is the sign for not and can be used in many places. The not sign is especially valuable when evaluating Boolean situations. Boolean variables can use the not sign and require less written code. The not sign is most often used to see if a Boolean variable is false.

Purpose:

Upon completing this project, you will be able to use a not sign to shorten the code in a Boolean situation.

Steps for Completion:

1. Open the **3 If-cart-evaluation.htm** file from the Music3 Student folder.
2. Find the openCheckout function near the bottom of the page.
3. Replace the code, `checkoutExists == false`, with code that uses the not operator to test to see if the `checkoutExists` variable is indeed false.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Click the Checkout button. The checkout window should open.
7. Click the Close Checkout button. The checkout window should close.

Project Details

Project Files:

3 If-cart-evaluation.htm

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 3

Topic: Logical and Comparison Operators

Subtopic: Not

Objectives Covered:

3 Decisions and Loops

3.1 Evaluate expressions that use logical and comparison operators
3.1.f !

LearnKey

And vs. Or

The and operator evaluates multiple expressions at the same time. The and symbol is `&&` when used in JavaScript code. Both conditions need to be true for the and evaluator to return true.

The or evaluator needs only one of the expressions being evaluated to be true to act. The or symbol is `||` when used in JavaScript code. This symbol is created by holding the Shift key and pressing the Backslash key twice.

Purpose:

Upon completing this project, you will be able determine the two expressions that need to return true for an and evaluator to return a set message.

Steps for Completion:

1. Open the **3lg-tuningkits-andor.htm** file from the Music3 Student folder.
2. Within the `updateList` function, replace the comment, `// and`, with the and operator.
3. Replace the comment, `// or`, with the or operator.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Select both the Wedge mute and Papps mute check boxes.
7. Click the Update List button. You should see a message stating that both mutes are selected.
8. Clear the Wedge mute check box.
9. Click the Update List button. You should see a message stating that one mute is selected.

Project Details

Project Files:

3lg-tuningkits-andor.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 3

Topic: Logical and Comparison Operators

Subtopic: And; Or

Objectives Covered:

3 Decisions and Loops

3.I Evaluate expressions that use logical and comparison operators

3.I.h `&&`

3.I.i `||`

If-Else

In programming, there are many situations in which a condition needs to be checked and valued at either being true or being false. As is the case with many other programming languages, JavaScript uses an if statement to check a condition. One or more actions can be taken if a condition is true. If a condition is false, an else statement is then used to take one or more different actions.

Purpose:

Upon completing this project, you will know how to create an if statement and how the if statement functions.

Steps for Completion:

1. Open the **32a-cart-evaluation.htm** file from the Music3 Student folder.
2. Scroll down and find the openCheckOut function.
3. Replace the comment, // add the conditional keyword, with the keyword needed to check to see if the condition is true.
4. Below the comment on consolidating the two functions, rework the closeCheckOut function so that it does the following:
 - a. Becomes part of the openCheckOut function.
 - b. Becomes the action that runs if the condition checked in the openCheckOut function is not true.
5. Remove any curly brackets and other punctuation no longer needed.
6. Rename the openCheckOut function to the word, **checkOut**
7. Save your changes.
8. Preview the webpage in a web browser.
9. Click the Checkout button. The checkout window should open.
10. Click the Close Checkout button. The checkout window should close.

Project Details

Project Files:

32a-cart-evaluation.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 3

Topic: Decision Statements

Subtopic: If

Objectives Covered:

3 Decisions and Loops

3.2 Complete and debug decision statements

3.2.a Single alternative (if)

In the previous project, you built an if-else statement, which checked a single condition and performed an action if it was true and a different action if it was false. Sometimes, more than one condition needs to be checked. An else if statement can be used to check if a condition is true after a previous condition has been established as false. You can have many else if portions within an overall if statement.

With any form of an if statement, the code leaves the if statement as soon as a condition returns true, no matter how many else if statements exist within the if-else block.

Purpose:

Upon completing this project, you will be able to use an else if function to add another action to an overall if statement.

Steps for Completion:

1. Open the **32b-tuningkits-andor.htm** file from the Music3 Student folder.
2. Within the `updateList` function, replace the comment, `// check this condition if the first one is false`, with the key phrase that will check this condition if the previous condition is false.
3. Replace the comment, `// do this if both conditions are false`, with the keyword used to take action if both conditions above are false.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Select the Wedge mute and Papps mute check boxes.
7. Click the Update List button. You should see a message indicating that both mutes are checked.
8. Clear the Wedge mute check box.
9. Click the Update List button. You should see a message indicating the one mute is checked.
10. Clear the Papps mute check box.
11. Click the Update List button. The message should disappear.

Project Details

Project Files:

32b-tuningkits-andor.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 3

Topic: Decision Statements

Subtopic: Else If

Objectives Covered:

3 Decisions and Loops

3.2 Complete and debug decision statements

3.2.b Dual alternative (if else)

Switch

The switch keyword is used when multiple conditions need different actions taken. Each condition within a switch statement is represented with the case keyword. The equivalent of an else keyword within a switch statement is the default keyword. Unlike the if statement, JavaScript needs to be told to exit the switch statement if the case condition is met. For this reason, after every case, a break statement needs to be made so that the block is exited from after a condition is met.

Purpose:

Upon completing this project, you will be able to create a switch statement with multiple conditions to be tested.

Steps for Completion:

1. Open the **32c-cart-switch.htm** file from the Music3 Student folder.
2. Find the showSpecialDiscount function.
3. Within the showSpecialDiscount function, replace the //check comment with the keyword used to start the statement that checks for multiple conditions.
4. Replace each //when comment with the keyword used for a condition within the statement you created in the previous step.
5. Replace each //exit comment with the keyword used to exit the statement.
6. Replace the //else comment with the keyword used for all other conditions within the statement.
7. Save your changes.
8. Preview your webpage in a web browser.
9. Click the Update cart button. If the current date is the 1st, 7th, or 15th of the month, you will see a special message. Otherwise, you will see a message to keep checking back for specials.

Project Details

Project Files:

32c-cart-switch.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 3

Topic: Decision Statements

Subtopic: Switch

Objectives Covered:

3 Decisions and Loops

3.2 Complete and debug decision statements

3.2.c Multiple alternative (switch)

LearnKey

Nested If

A nested if statement is an if statement inside of an if statement. The nested if statement is normally used when checking for one or more conditions inside a condition that is found to be true.

Purpose:

Upon completing this project, you will be able to nest an if statement that will return different messages when set conditions are met.

Steps for Completion:

1. Open the **32d-tuningkits-andor.htm** file from the Music3 Student folder.
2. Find the `updateList` function.
3. Within the `updateList` function, add an if statement at the point of the comment that begins with, `//` nest an if statement, that will do the following:
 - a. If the Wedge mute check box is selected, return the message indicating as such.
 - b. If the Papps mute check box is selected, return the message indicating as such.
4. Remember to add all appropriate punctuation.
5. Save your changes.
6. Preview the webpage in a web browser.
7. Select the Wedge mute check box.
8. Click the Update List button. You should see a message indicating that the Wedge mute check box is selected.
9. Clear the Wedge mute check box.
10. Select the Papps mute check box.
11. Click the Update List button. You should see a message indicating that the Papps mute check box is selected.
12. Close all of your open files, both in the browser and in your code editor.

Project Details

Project Files:

32d-tuningkits-andor.htm

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain 3

Topic: Decision Statements

Subtopic: Nested If

Objectives Covered:

3 Decisions and Loops

3.2 Complete and debug decision statements

3.2.d Nested If

For

The for loop runs a block of code a set number of times. The run times can be set by us or by a value. The for loop has three arguments: a starting point, the loop stop point, and the number to increase or decrease each time the loop is run.

Purpose:

Upon completing this project, you will be able to create and use a for loop to run a block of code a set number of times.

Steps for Completion:

1. Open the **33a-tuningkits-loops.htm** file from the Music3 Student folder.
2. Find the `updateList` function.
3. Within the `updateList` function, replace the comment, `// start the for loop`, with the opening statement in a for loop to run through the array of the `tuningList` variable. The loop should have the following:
 - a. A variable named `i` that starts at 0.
 - b. The length of the loop should be the length of the `tuningList` array.
 - c. The loop should increment by one each time the loop runs.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Select the Papps mute check box.
7. Click the Update List button. The two tuning items and the Papps mute item should display as a bulleted list.

Project Details

Project Files:

33a-tuningkits-loops.htm

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 3

Topic: Loops

Subtopic: For

Objectives Covered:

3 Decisions and Loops

3.3 Complete and debug loops

3.3.a For

3.3.b For in

Do and While Loops

While the for loop runs a set number of times, do and while loops run based on a condition, specifically until a condition is met. The difference between the two loops is simple: a while loop is not guaranteed to run as it will not run when a condition has already been met. A do loop will run at least once.

When using a while loop, it is imperative to make sure a condition is eventually met. Otherwise, you will have code that is stuck in an endless loop.

Purpose:

Upon completing this project, you will be able to create a while loop, a do loop, and you will understand the difference between the two.

Steps for Completion:

1. Open the **33b-practice-loops.htm** file from the Music3 Student folder.
2. Within the lessonSchedule function, replace the comment, // add condition here, with a conditional loop to run so long as the lessonStartDate is less than the lessonEndDate.
3. Save your changes.
4. Preview the webpage in a web browser.
5. Click in the Enter the number of hours per week field.
6. Type: **4** and press the Enter key. You should see a list of lesson dates.
7. Return to the code editor.
8. Adjust the syntax of the loop so that it runs at least once.
9. Save your changes.
10. Refresh the webpage in the web browser.
11. Click in the Enter the number of hours per week field.
12. Type: **4** and press the Enter key. You should still see a list of lesson dates.

Project Details

Project Files:

33b-practice-loops.htm

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain 3

Topic: Loops

Subtopic: While; Do

Objectives Covered:

3 Decisions and Loops

3.3 Complete and debug loops

3.3.c While

3.3.d Do while

Break and Continue

Earlier in this domain, you saw how a break statement is used to exit a block of code, specifically within a switch statement. You may need to do this within a loop as well, as a loop may reach a condition to where the loop needs to be exited.

For situations in which you do not need to exit a loop but rather you want to skip over the code within the iteration of a loop, use a continue statement.

Purpose:

Upon completing this project, you will be able to break code in a loop within an if statement.

Steps for Completion:

1. Open the **33d-practice-break.htm** file from the Music3 Student folder.
2. Replace the comment, // add condition, with a condition checking to see if the lessonStartDate equals the breakDate.
3. Replace the comment, // exit the loop if the condition is true, with the keyword needed to exit the loop.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Click in the Enter the number of hours per week field.
7. Type: **2** and press the Enter key. You should see no lessons after 11/20.
8. Return to your code editor.
9. Change the keyword used within the if statement to have the loop skip over the date if the condition is true but not exit the loop completely.
10. Save your changes.
11. Preview the webpage in a web browser.
12. Click in the Enter the number of hours per week field.
13. Type: **2** and press the Enter key. You should see no lesson on 11/27 but a lesson on 12/4.
14. Close all of your open files in your browser and your code editor.

Project Details

Project Files:

33d-practice-break.htm

Estimated Completion Time:

15 minutes

Video Reference:

Domain 3

Topic: Loops

Subtopic: Break; Continue

Objectives Covered:

3 Decisions and Loops

3.3 Complete and debug loops

3.3.e Break

3.3.f Continue

JavaScript

Domain 4

Presented By
LK LearnKey®

Fill-in-the-Blanks Domain 4

Instructions: While watching Domain 4, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

DOM Tree

1. A _____ always starts with a Window project. [Window]
2. The document, every HTML element, and every _____ descends from this document object. [Document]
3. In the DOM, the two tags inside of the HTML tag are _____ and body. [Body]
4. For the exam, know the starting point of a DOM and know how to _____ a DOM. [Other HTML Elements]

HTML Events

5. The onchange event fires when an _____ is changed. [Onchange]
6. On objects, actions are often triggered through _____, [Onmouseover]
7. With an onload event, action is taken when an HTML _____ loads. [Onload]
8. The two main actions taken as a result of an onclick event are a _____ runs, or _____ within the event runs. [Onclick]
9. The onmouseout event triggers action when a mouse is moved _____ from an object. [Onmouseout]
10. The onkeydown event triggers action when an item on the _____ is pressed. [Onkeydown]

Output to HTML Documents

11. The innerHTML property allows us to control the text in between the opening and closing tag of an existing HTML element _____. [innerHTML]
12. The document.write method allows us to write both HTML elements and _____ to a webpage. [Document.Write]

Locate, Modify, and Add Elements

13. Once you get an element's ID, you can change any of its _____ using JavaScript. [GetElementById]
14. The getElementsByTagName method will list elements and their _____. [getElementsByTagName]
15. The classlist. _____ property shows the number of times a class is being used. [getElementsByName]
16. The first item in the array is item _____. [SetAttribute]
17. We can use the createElement method to place a new HTML element in a _____ place on a webpage. [createElement]

A window is a browser window that a user will interact with online. The window object contains properties such as height and width. The window object also contains objects such as the document and all objects contained within a document. The basic structure of a window and its objects is known as the DOM (Document Object Model).

Purpose:

Upon completing this project, you will understand objects and their properties, how they relate to each other, and how they are used on a webpage. You will be able to start a DOM.

Steps for Completion:

1. Name some properties of a Window object, besides height and width:
2. Open the **41a-cart-DOM.htm** file from the Music4 Student folder.
3. Find the checkoutWindow function.
4. Within the checkoutWindow function, add code to the window.open method to hide the status bar.
5. Save your changes.
6. Preview the webpage in a web browser.
7. Click the Checkout button. A checkout window should open and should not display a status bar.
8. Click the Close Checkout button. The checkout window should close.

Project Details

Project Files:

41a-cart-DOM.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 4

Topic: DOM Tree

Subtopic: Window

Objectives Covered:

4 Document Object Model

4.1 Identify and construct the Document Object Model (DOM) tree

4.1.a Window

Document, Body, and Other HTML Elements

The document is a page of HTML or JavaScript code that will style a webpage in a browser. The body is where the content code is found for an HTML document.

Purpose:

Upon completing this project, you will be able to identify a document and its main parts.

Steps for Completion:

1. Open the **41b-tuningkits-DOM.htm** file from the Music4 Student folder.
2. What is the first group of elements in the document?
3. What two tags are in the <html>element?
4. The head tag has which tags?
5. The link tag has what reference?
6. What are the sections of the body element?
7. Where would code for a paragraph tag be found?

Project Details

Project Files:

41b-tuningkits-DOM.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 4

Topic: DOM Tree

Subtopic: Document; Body; Other HTML Elements

Objectives Covered:

4 Document Object Model

4.1 Identify and construct the Document Object Model (DOM) tree

4.1.b Document

4.1.c Body

4.1.d Other HTML elements

The onchange event usually happens when an object is immediately changed by an action. What sets this out as an event is that the user is interacting with the code through a keypress or mouse click. Or, an onchange event can occur through a webpage loading or through an object changing a property, such as its value.

Purpose:

Upon completing this project, you will know how an onchange event is used in JavaScript.

Steps for Completion:

1. Open the **42a-practice-DOM.htm** file from the Music4 Student folder.
2. Replace the HTML comment, `<!-- call the getPracticeHours function when this field is changed -->`, with a call to the `getPracticeHours` function when the field changes.
3. Save your changes.
4. Preview the webpage in a web browser.
5. Click in the Enter the number of hours field.
6. Type: **4** and press the Tab key. Notice the messages that appear as a result of your entering the number of hours in a field.

Project Details

Project Files:

42a-practice-DOM.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 4

Topic: HTML Events

Subtopic: Onchange

Objectives Covered:

4 Document Object Model

4.2 Identify and handle document, form, keyboard, and mouse events

4.2.d Onchange

Onmouseover

The onmouseover event triggers action when a user rolls a mouse over an object. A common use of this is the display of an image changing when a user rolls a mouse over it.

Purpose:

Upon completing this project, you will be able to create an onmouseover event that will return a larger image in a separate image element.

Steps for Completion:

1. Open the **42b-products-DOM.htm** file from the Music4 Student folder.
2. Find the mouseOverPic function near the bottom of the page. Notice that the function uses a parameter for the image display.
3. Scroll back up to the section on thumbnails.
4. Replace the HTML comment, `<!-- rollover mandolin-->`, with a call to the mouseOverPic function, using mandolin as a parameter, when the mouse rolls over the link.
5. Replace the HTML comment, `<!-- rollover trombone-->`, with a call to the mouseOverPic function, using trombone as a parameter, when the mouse rolls over the link.
6. Replace the HTML comment, `<!-- rollover drum-->`, with a call to the mouseOverPic function, using drum as a parameter, when the mouse rolls over the link.
7. Replace the HTML comment, `<!-- rollover kazoo-->`, with a call to the mouseOverPic function, using kazoo as a parameter, when the mouse rolls over the link.
8. Replace the HTML comment, `<!-- rollover musicbooks-->`, with a call to the mouseOverPic function, using musicbooks as a parameter, when the mouse rolls over the link.
9. Save your changes.
10. Preview the webpage in a web browser.
11. Roll your mouse over each of the thumbnail images. A larger version of the image should display as you roll over each thumbnail.

Project Details

Project Files:

42b-products-DOM.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 4

Topic: HTML Events

Subtopic: Onmouseover

Objectives Covered:

4 Document Object Model

4.2 Identify and handle document, form, keyboard, and mouse events

4.2.i Onmouseover

Onload and Onclick

The onload event will return an action on the loading of an HTML element. The onload event is often used to call a function when a webpage loads.

An onclick event happens when an object or button is clicked and a code runs. The onclick event is the most frequently used event in JavaScript. The two main actions of an onclick event are that either a function or a code within the event will run.

Purpose:

Upon completing this project, you will be able to create an onload event that will give an alert when a webpage is loaded.

Steps for Completion:

1. Open the **42c-cart-DOM.htm** file from the Music4 Student folder.
2. In the body tag, create an onload event that will create an alert with the text: **This is an example of the onload event.**
3. Find the div tag with the class, itemquantity and the HTML comments, <!-- run when clicked-->.
4. Within that div tag, replace each HTML comment with the event that triggers when each item is clicked.
5. Save your changes.
6. Preview the webpage in a browser. An alert with the message you delivered should appear.
7. Close the alert.
8. Click the +, -, Clear, and Add Five buttons for the first two instruments. These buttons should, in order, add one to the quantity, subtract one from the quantity, clear the quantity, and set the quantity to five.

Project Details

Project Files:

42c-cart-DOM.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 4

Topic: HTML Events

Subtopic: Onload; Onclick

Objectives Covered:

4 Document Object Model

4.2 Identify and handle document, form, keyboard, and mouse events

4.2.a Onload

4.2.h Onclick

Onmouseout

The onmouseout event is triggered and returns an action when a mouse moves away from an object. This is often used to display a message or change an object when a mouse moves away from an object.

Purpose:

Upon completing this project, you will be able to create an onmouseout event that will guide the user on page navigation.

Steps for Completion:

1. Open the **42e-cart-mouse.htm** file from the Music4 Student folder.
2. Find the code for the div tag with the order_buttons class.
3. Replace the HTML comment, `<!--add instructions to click the button when mouse moves away-->`, with an alert function with the text, **Click the Update Cart button to update the order total**, that will occur when one moves the mouse away from the Update Cart button.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Hover the mouse over the Update Cart button and then move away from the Update Cart button. You should see an alert with instructions on how to use the Update Cart button.

Project Details

Project Files:

42e-cart-mouse.htm

Estimated Completion

Time:

5-10 minutes

Video Reference:

Domain 4

Topic: HTML Events

Subtopic: Onmouseout

Objectives Covered:

4 Document Object Model

4.2 Identify and handle document, form, keyboard, and mouse events

4.2.j Onmouseout

LearnKey Onkeydown

The onkeydown event will return an action if any key on the keyboard is pressed. This event is often geared toward users who would rather not use a mouse in order to trigger action. In addition, the event could be used to push users toward using a mouse. A similar event, onkeyup, fires when a key on the keyboard is released.

Purpose:

Upon completing this project, you will be able to create an onkeydown event.

Steps for Completion:

1. Open the **42f-cart-mouse.htm** file from the Music4 Student folder.
2. Find the HTML comment, `<!-- encourage mouse use -->`
3. Replace the HTML comment that will, when a key is pressed down, display an alert with this text: **It may be easier to use buttons to change this field.**
4. Save your changes.
5. Preview the webpage in a web browser.
6. Click in the Quantity field of the Mandolin and type: **5**. You should see an alert explaining that it may be easier to use buttons to change the field.
7. Close your open files, both in the browser and in your code editor.
8. Save the file as **42f-cart-mouse-finished.htm**

Project Details

Project Files:

42f-cart-mouse.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 4

Topic: HTML Events

Subtopic: Onkeydown

Objectives Covered:

4 Document Object Model

4.2 Identify and handle document, form, keyboard, and mouse events

4.2.e Onkeydown

InnerHTML

The innerHTML property allows the text between the opening and closing tags to be dynamically controlled in an existing HTML element. It is important to know when to use the equal sign to replace content or when to use the plus equal signs to append or add an attachment to existing content. Changing the innerHTML property allows the content of a page to be updated without the need to refresh the page.

Purpose:

Upon completing this project, you will know when to change an element's innerHTML property.

Steps for Completion:

1. Open the **43a-practice-DOM.htm** file from the Music4 Student folder.
2. Within the `getPracticeHours` function, replace both instances of the comment, // add code to set the element text, with the property and operator necessary to set the text of each HTML element.
3. Within the `lessonSchedule` function, replace both instances of the comment, // add code to append text to this element, with the property and operator necessary to add the subsequent text to the existing text within each HTML element.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Click in the Enter the number of hours per week field.
7. Type: **10** and press the Enter key. You should see a message indicating when classes start. A total of two class dates should be displayed.

Project Details

Project Files:

43a-practice-DOM.htm

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 4

Topic: Output to HTML Documents

Subtopic: innerHTML

Objectives Covered:

4 Document Object Model

4.3 Complete and debug code that outputs to an HTML document

4.3.b innerHTML

Document.write

The document.write method allows you to write text and HTML elements dynamically to a webpage. The document.write method will overwrite existing content, so this is best to use on a webpage with no content.

Purpose:

Upon completing this project, you will be able to use the document.write method to add content dynamically to a webpage.

Steps for Completion:

1. Open the **43b-cart-DOM.htm** file from the Music4 Student folder.
2. Find the checkoutWindow function near the bottom of the page.
3. Replace the comment, // add text to the new window, with a statement that adds the text, **This document is current as of** plus the current date. Make sure this is placed inside of a paragraph tag.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Click the Checkout button. A small checkout window should appear with the message you configured in this project.
7. Click the Close Checkout button to close the window.
8. Close all of your open files in your editor and web browser.

Project Details

Project Files:

43b-cart-DOM.htm

Estimated Completion Time:

10 minutes

Video Reference

Domain 4

Topic: Output to HTML Documents

Subtopic: Document.write

Objectives Covered:

4 Document Object Model

4.3 Complete and debug code that outputs to an HTML document

4.3.a Document.write

GetElement

There are three ways in which JavaScript works with elements to set properties and run methods. They are:

getElementById: elements are coded through using the element ID. Using an element ID is the easiest way to locate an element in JavaScript. It is a good practice to assign an ID to all HTML elements that can be used in JavaScript code. It is important to have a consistent set of rules for creating ID names for your elements.

getElementsByTagName: a method that allows you to get all the elements of a HTML tag. In most cases, this is used for testing purposes, so you will want to list how many elements are in the document. A `getElementsByTagName` method stores a set of values in an array.

getElementsByClassName: a method that allows you to know what classes are in the document, and how many times a class is used. This will help in determining if a class attribute should be changed.

Purpose:

Upon completing this project, you will know how to use `getElementById`, `getElementsByTagName`, and `getElementsByClassName`.

Steps for Completion:

1. Open the **44a-comparison-get.htm** file from the Music4 Student folder.
2. Replace the comment, `// get all of the images`, with a method used that will get all instances of the `img` tag.
3. Replace the comment, `// get all of the social class instances`, with a method that will get all instances of the social class.
4. Replace the comment, `// get sitelogo element`, with the method that will get the element with an ID of `sitelogo`.
5. Save your changes.
6. Preview the webpage in a web browser.
7. Click the Get Images button. A list of images for the webpage should display.
8. Click the Get Social Classes button. A number 1 should appear, indicating that the social class is used once on the page.

Project Details

Project Files:

44a-comparison-get.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 4

Topic: Output to HTML Documents

Subtopic: `getElementById`;
`getElementsByTagName`;
`getElementsByClassName`

Objectives Covered:

4 Document Object Model

4.4 Complete and debug code that locates, modifies, and adds HTML elements and attributes to documents

4.4.a `getElementById`

4.4.b `getElementsByTagName`

4.4.c `getElementsByClassName`

SetAttribute and CreateElement

The `setAttribute` method allows for a dynamic change of an element's attributes such as a class on an element through an onclick event. This is often used to allow a user to change the style of an element to make it more readable.

The `createElement` method allows an element to be dynamically created and placed in a specific spot on a webpage. The new element is placed within an element that has an ID and can be called through the `getElementById` method. Once an element is created, one way in which text can be added to it is through the `createTextNode` method, which creates content to be added to the new element, and then the content is added through the `appendChild` method.

A user can also interact with a webpage using event handlers and listeners in JavaScript. An event refers to actions by the user or actions by the browser. Some common events include 'onload', 'onclick', and 'onchange'. The 'onclick' event is commonly used for buttons or input fields, while the 'onload' event is commonly used on a webpage's body tag to execute code when a webpage has finished loading.

An event listener or event handler specifies the event it is listening for and the action (function) that will take place should that event occur. For example, an event listener can listen for a click on an HTML element and then run a function. That function can then retrieve data, like the current date, and insert that data into other HTML elements or JavaScript code.

Purpose:

Upon completing this project, you will be able to use the `setAttribute` method to allow the user to change a heading style.

Steps for Completion:

1. Open the **44d-comparison-get.htm** file from the Music4 Student folder.
2. In the `setAltHeadingClass` function, replace the comment, `// set the class to altheading`, with the code necessary to change the class of the element in the function to `altheading`.
3. In the `addConstructionMsg` function, do the following:
 - a. Replace the comment, `// create a paragraph element`, with the code necessary to create a new paragraph element.
 - b. Replace the comment, `// add this text node`, with the method used to add the subsequent text as a new text node.
 - c. Replace the comment, `// add text node to variable`, with the code necessary to add the `constructionText` content to the `constructionP` variable.
 - d. Replace the comment, `// add variable to element`, with the code necessary to add the `constructionP` element to the element with an ID of `sitelogo`.
4. Save your changes.
5. Preview your webpage in a web browser.
6. Click the Change Heading button. The style of the heading should change.
7. Click the Create Element button. A new text message should appear on the page.
8. Close all of your open files in your code editor and your web browser.

Project Details

Project Files:

44d-comparison-get.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 4

Topic: Output to HTML Documents

Subtopic: `setAttribute`;
`createElement`

Objectives Covered:

4 Document Object Model

4.4 Complete and debug code that locates, modifies, and adds HTML elements and attributes to documents

4.4.d `setAttribute`

4.4.e `createElement`

4.5 Create events using event handlers and listeners

4.5.a DOM events

4.5.b HTML attribute event

4.5.c `addEventListener`

JavaScript

Domain 5

Presented By
LK LearnKey®

Fill-in-the-Blanks Domain 5

Instructions: While watching Domain 5, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

Retrieve Input and Set Form Field Values

1. Input tags with _____ typically do not have innerHTML text. [Retrieve Form Values]
2. We can get a value of an individual field at any time through the _____ method. [DOM Path]
3. Though this is a different type of element, we are still using the _____ attribute to retrieve what was chosen. [Values from Elements]
4. Prepopulating a field is a good practice for when a field _____ is the same for virtually every entry. [Prepopulating Values]
5. Setting an input field to a _____ type field will mask the field. [Masking Values]

Input Validation

6. With _____ fields, you may want consistent casing no matter how data is entered. [Upper and Lower Case]
7. JavaScript is a _____ language, even for values in form fields. [String Comparisons]
8. Some form validation can take place without _____, as simple HTML can be used instead. [Validation]
9. In JavaScript NaN means Not a _____. [NaN]

Form Submission Process

10. You can add multiple _____ to an onsubmit event to validate a form. [Onsubmit]
11. When a form is processed, a _____ for processing the form data is specified, right in the form tag. [Understand Post vs. Get]
12. The target is defined in the _____ attribute of the form tag. [Understand Potential Submission Targets]

Retrieving Form Values

When a form is created, the information that is filled out will need to be accessed. An input tag does not have the innerHTML attribute, so the value attribute is used to retrieve the form information that was entered in an input field.

Purpose:

Upon completing this project, you will be able to use the value attribute to retrieve the first and last name entered in an input field.

Steps for Completion:

1. Open the **51a-contact.htm** file from the Music5 Student folder.
2. Within the fillTestResults function, replace the comment, // get what the user typed in for a first name, with a method that will retrieve what the user typed into the firstname input field in the form.
3. Replace the comment, // get what the user typed in for a last name, with a method that will retrieve what the user typed into the lastname input field in the form.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Enter a first name and last name in the First and Last name fields.
7. Click the Test form button. You should see a message with the name display.

Project Details

Project Files:

51a-contact.htm

Estimated Completion

Time:

5-10 minutes

Video Reference:

Domain 5

Topic: Retrieve Input and Set Form Field Values

Subtopic: Retrieve Form Values

Objectives Covered:

5 HTML Forms

5.1 Complete and debug code that retrieves form input and sets form field values

5.1.a Retrieve form values

LearnKey DOM Path

JavaScript can access forms or form elements directly with the DOM path. When accessing forms in the Console, `document.forms` is an array that will start with the number zero. Each form has an array of elements, through the `document.forms.elements` property, with the array of elements for each form starting at zero.

Purpose:

Upon completing this project, you will be able to follow the DOM path in the Console to pull up elements and values.

Steps for Completion:

1. Open the **51b-contact.htm** file from the Music5 Student folder in a web browser.
2. Open the Console.
3. Use the Console to list all the forms on this webpage. How many forms are on the webpage?
4. What is the ID of the first form?
5. Use the Console to list all of the elements on the first form of the page. How many elements are on the form?
6. How many input elements are on the form?

Project Details

Project Files:

51b-contact.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 5

Topic: Retrieve Input and Set Form Field Values

Subtopic: DOM Path

Objectives Covered:

5 HTML Forms

5.1 Complete and debug code that retrieves form input and sets form field values

5.1.b Identify the DOM path

Values from Elements

When a form is created, the information that is filled out will need to be accessed. Forms are made of input fields and elements and a value is typed into a field, selected from a drop-down list, or assigned through one or more options, such as a single radio button or one or more check boxes.

Purpose:

Upon completing this project, you will be able to retrieve a value from an element and access the selected information.

Steps for Completion:

1. Open the **51c-contact.htm** file from the Music5 folder.
2. In the fillTestResults function, following the firstName and lastName variables, create a variable named **found** that will contain the value from the found field.
3. Add, to the testResults.innerHTML property, the text: **You found us via**, a space, the found variable, and a period.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Enter a first and last name and select a field for the How did you hear about us field.
7. Click the Test form button. You should see a message on the page with your name and the choice you made in the found field.

Project Details

Project Files:

51c-contact.htm

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 5

Topic: Retrieve Input and Set Form Field Values

Subtopic: Values from Elements

Objectives Covered:

5 HTML Forms

5.1 Complete and debug code that retrieves form input and sets form field values

5.1.c Get values from different types of elements

Prepopulating Values

A prepopulated field will require less input for a user. The field will need to give the same exact answer in every instance to set a prepopulated answer. A field is often prepopulated through the use of an if or if-else statement.

Purpose:

Upon completing this project, you will be able to set a field to prepopulate an answer based on an entry in another field.

Steps for Completion:

1. Open the **5Id-contact.htm** file from the Music5 Student folder.
2. Within the fillState function, replace the comment, // add city and state logic, with code that will do the following:
 - a. Assign a value of UT to the state variable if someone enters Salt Lake City for a location.
 - b. Assign a value of NV to the state variable if someone enters Summerlin for a location.
 - c. Leaves the state variable blank if any other city value is chosen.
3. Within the city element, call the fillState function when an option is chosen, using the city indicated as a parameter for each option with a city.
4. Save your changes.
5. Preview the webpage in a web browser.
6. Select Salt Lake City for a location. The state should change to UT.
7. Select Summerlin for a location. The state should change to NV.

Project Details

Project Files:

5Id-contact.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 5

Topic: Retrieve Input and Set Form Field Values

Subtopic: Prepopulating Values

Objectives Covered:

5 HTML Forms

5.1 Complete and debug code that retrieves form input and sets form field values

5.1.d Prepopulate values

Masking Values

Sensitive information such as Social Security numbers can make a user nervous to enter the information into an input field, so it is a best practice to mask the sensitive values being entered. When masking a value, the value will be replaced with dots or another symbol instead of the value being entered. A password type field will mask a field.

Purpose:

Upon completing this project, you will be able to mask a field value using a password type field.

Steps for Completion:

1. Open the **51e-contact.htm** file from the Music5 Student folder.
2. In the password field code on line 104 of the code, change the type from text to **password**
3. Save your changes.
4. Preview the webpage in a web browser.
5. Begin to type in the password field. You should see masked characters.
6. Close all of your open files in your code editor and web browser.

Project Details

Project Files:

51e-contact.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 5

Topic: Retrieve Input and Set Form Field Values

Subtopic: Masking Values

Objectives Covered:

5 HTML Forms

5.1 Complete and debug code that retrieves form input and sets form field values

5.1.e Mask values

Uppercase and Lowercase

JavaScript is a case-sensitive language. There will be instances where consistent casing will be needed in a form field. Text can be converted to its uppercase or lowercase equivalent at any time using the `toUpperCase` and `toLowerCase` methods, respectively. This is a common tactic used to compare strings to a condition. As an example, if you are looking for a match to a city, such as Springfield, you could convert `springfield`, `Springfield`, or `SPRINGFIELD` to all lowercase or all uppercase and then use that value in a condition.

Purpose:

Upon completing this project, you will be able to set the text of a form field to return in uppercase or lowercase and then use these in a conditional statement.

Steps for Completion:

1. Open the **52a-contact.htm** file from the Music5 Student folder.
2. Within the `convertUpper` function, replace the comment, `// convert to upper`, with the code necessary to convert the text variable to an uppercase value.
3. Within the `convertLower` function, replace the comment, `// convert to lower`, with the code necessary to convert the text variable to a lowercase value.
4. Within the `checkCity` function, replace the comment, `// make the city variable uppercase`, with the code necessary to set the city variable to an uppercase version of itself.
5. Save your changes.
6. Preview the webpage in a web browser.
7. Click in the State field.
8. Type **nv** and press the Tab key. Notice that the value converts to uppercase.
9. Click in the City field.
10. Type **las vegas** and press the Tab key. A message about the Summerlin location should appear at the bottom of the page.

Project Details

Project Files:

52a-contact.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 5

Topic: Input Validation

Subtopic: Upper and Lower Case;
String Comparisons

Objectives Covered:

5 HTML Forms

5.2 Complete and debug code
that performs input validation

5.2.a Case

5.2.b String comparisons

Validation

Valid data needs to be entered on a form before the form is submitted. The two tasks of validation are to check that required fields have data/value entered and that the data/value entered is a legitimate value. There are form validations that do not require JavaScript, such as an email field.

Purpose:

Upon completing this project, you will be able to add validations to fields.

Steps for Completion:

1. Open the **52c-contact.htm** file from the Music5 Student folder.
2. Make the input element with an ID of zip a required field.
3. Add a call to the checkFields function within the button with the name, testForm.
4. Within the checkFields function, replace the comment, // add operator, with the operator necessary to check and make sure neither the firstName nor lastName fields are empty.
5. Save your changes.
6. Preview the webpage in a web browser.
7. Click the Test Form button. You should see a message indicating that one or more required name fields is empty.
8. Fill out the First Name, Last Name, and Zip fields.
9. Click the Test Form button. A message should appear at the bottom of the form.

Project Details

Project Files:

52c-contact.htm

Estimated Completion Time:

10 minutes

Video Reference:

Domain 5

Topic: Input Validation

Subtopic: Validation

Objectives Covered:

5 HTML Forms

5.2 Complete and debug code that performs input validation

5.2.d Not blank

NaN means Not a Number and will display when a calculation that expects a number takes place, but the calculation return is not a number. The written form of a number is not used in calculations, so fields that make calculations should be set to require a number input. The NaN indicator can be corrected by validation or user training.

Purpose:

Upon completing this project, you will better understand the cause of a NaN message to appear.

Steps for Completion:

1. Open the **52d-practice-DOM.htm** file from the Music5 Student folder in a web browser.
2. What does NaN stand for?
3. Click in the Enter the number of hours per week field.
4. Type **three** and press the Enter key. Does NaN appear in the message?
5. What is the cause of the NaN indicator?
6. What can be done to correct a NaN?
7. Close all open files in your browser and code editor.

Project Details

Project Files:

52d-practice-DOM.htm

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 5

Topic: Input Validation

Subtopic: NaN

Objectives Covered:

5 HTML Forms

5.2 Complete and debug code that performs input validation

5.2.c NaN

Onsubmit

The onsubmit event is used to run the last code before a form submits data to be processed. The most common use of the onsubmit event is the use of one or more functions used to validate a form. One onsubmit event can use multiple functions.

Purpose:

Upon completing this project, you will be able to create an onsubmit event that uses existing functions to validate form fields on a form.

Steps for Completion:

1. Open the **53a-contact.htm** file from the Music5 Student folder.
2. In the opening form tag, replace the comment, `<!-- add code to run checkFields-->`, with code that will run the checkFields function when the form is submitted.
3. Save your changes.
4. Preview the webpage in a web browser.
5. Click the Send It button. You should get a message indicating that the zip field is required.

Project Details

Project Files:

53a-contact.htm

Estimated Completion Time:

5 minutes

Video Reference:

Domain 5

Topic: Form Submission Process

Subtopic: Onsubmit

Objectives Covered:

5 HTML Forms

5.3 Describe the form submission process

5.3.a Onsubmit

Post vs. Get and Submission Targets

The two methods used to process data are post and get. Post is used when information is posted in an area. Get is used when you are trying to get information, typically used for a search engine. The method for processing is in the form tag. The get method will publish terms a user enters in the URL, so do not use this method if data needs to be kept secure.

Regarding potential submission targets for data on a form, there are three common targets:

Database: data can be added to a database, or, as in the case of a search engine, data can be used to look up a database to return matches.

Script: the data can be directed to a script file for further processing. The script file is a go-between for a webpage submitting data to a data source, such as a database. Many of these scripts are server-side scripts that use a variety of programming languages, such as C# or Python.

Webpage: the data can be processed on another webpage. That webpage could have direct access to a database.

Purpose:

Upon completing this project, you will understand the post and get methods and how to write them in code. You will also gain an understanding of the use of potential submission targets.

Steps for Completion:

1. Open the **53b-contact.htm** file from the Music5 Student folder.
2. In the opening form tag, replace the comment, `<!-- add submission method-->`, with code to indicate that this form will use the post method to process the data.
3. Save your changes.
4. Preview the webpage in a web browser.
5. Click the Send it button. You should see a message that the zip field, a required field, needs to be filled out.
6. Looking at this form, what would be the most likely submission target used for the data?
7. Close all of your open files in your browser and code editor.
8. Close your browser and your code editor.

Project Details

Project Files:

53b-contact.htm

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 5

Topic: Form Submission Process

Subtopic: Understand Post vs. Get;
Understand Potential Submission Targets

Objectives Covered:

5 HTML Forms

5.3 Describe the form submission process

5.3.b Post vs. get

5.3.c Potential targets for submission

JavaScript

Appendix

Presented By
LK LearnKey®

Array	An object in JavaScript which has multiple containers. This allows it to hold multiple values.
Assignment Operator	An operator which is used to set a variable to be equal to a string, number, Boolean operator, list, or date.
BOM	A Browser Object Model (BOM) is a browser-specific convention which refers to all the objects exposed by a web browser.
Boolean Variable	A type of variable with two possible values, which are true or false. It is often used to determine a course of action within a program.
Break	A statement used to stop or get out of a loop.
Case	A text option which can be set to upper or lower. JavaScript is a case-sensitive language, so code and function names must be consistent to work correctly.
Ceiling	A method used in JavaScript to move numbers to a higher whole number.
Comment	A tool used to explain what code does and to tell JavaScript to ignore code temporarily.
Comparison Operator	An operator which is used inside JavaScript to compare arguments.
Console.log	A debugging tool in which requests can be entered to get information about selected script.
Constant	A variable which cannot change throughout its use in code.
Continue	A keyword which is used to skip a turn in a loop where a user is displaying a message or running an action a set number of times.
createElement	A method which is used to create dynamically an element and place that element in a specific spot on a webpage.
Data Localization	The process of using the toLocalDateString method to retrieve a date displayed in a localized format.
Debug	To try to find errors in code and then fix them.
Decrement	An assignment operator which decreases a value by one.
Do Loop	A loop that always runs at least once and then checks the condition.
Document.write	A method which allows the user to write both HTML elements and text to a webpage.
DOM	The Document Object Model (DOM) is an application programming interface which defines the structure and accessibility of a document.
Else If	A statement which checks a condition to see if it is true or false, and then takes action based on the result.
Else Statement	A statement used in conjunction with if and elif statements. It provides an if all else fails option.
Finally	A keyword which is used to deliver a final message to a user regardless of whether an argument was successful or not.
Floor	A JavaScript method used to move numbers to a lower whole number.
For Loop	A loop which will act for a set number of times before canceling.
Get	A method used to process data when trying to get information.
getElementByClassName	A method used in JavaScript to locate elements by their class name.
getElementById	A JavaScript method which is used to locate elements by ID.
getElementsByTagName	A method which is used to locate elements by its tag name.
If	A statement used in JavaScript which checks a condition to see if it is true or false, and then takes action based on the result.
Increment	An assignment operator which increases a value by one.
Indentation	The process of moving code to the right in order to make it easier to read, but also to complete if statements.
Index Number	A tool which is used to access an element inside of an array.

Initialization	The act of storing a value in a variable in JavaScript.
innerHTML	A property in JavaScript which allows the user to control dynamically the text in between the opening and closing tag of an existing HTML element.
Instantiation	A tool used to create a new template for an object, otherwise known as a class.
Logical Operator	A symbol used in a logical statement to determine the logic between values. Examples would include &&, !, , and others.
Loop	A statement which allows code to be executed multiple times.
Masking Value	A tool used in JavaScript to mask or hide characters typed into sensitive fields on a form, such as a password field. Setting an input field to a password type field will mask the field.
Math.abs	A method used in JavaScript to make a number represent absolute value.
Max	A method used in JavaScript to retrieve the highest numbers from a range of numbers.
Method	A tool within JavaScript which is used to take action on objects and their properties.
Min	A JavaScript method which retrieves the lowest numbers from a range of numbers.
Modulus	The term used to indicate the remainder when dividing whole numbers in JavaScript.
NaN	Not a Number (NaN) is an indicator which will usually appear when a calculation expecting a number takes place.
Nested If	An if statement that is placed inside of another if statement.
Noscript	A tag which is used to ensure that JavaScript code works on a browser that does not support JavaScript.
Null	An empty variable on a webpage which has been declared and is defined as empty but not zero.
Number	A variable in JavaScript which is used to store data as floating point decimals.
Object	An item in JavaScript which can be manipulated.
Onchange	An event which executes when an object is changed.
OnClick	One of the most commonly used events in JavaScript, it triggers code when an object is clicked.
Onkeydown	An event which will trigger when a user presses a key on a keyboard.
Onload	An event in JavaScript which occurs when an HTML element loads.
Onmouseout	Similar to an onmouseover event, this event triggers when a mouse is moved away from an object.
Onmouseover	A JavaScript event which triggers when a user rolls a mouse over an object.
Onsubmit	The event used to run, or validate, any last code before a form sends its data to be processed.
Pop	Similar to push, it is a tool used to remove the last item from an array.
Post	A method used to process data when a user is posting information somewhere.
Pow	A type of method in JavaScript which takes a number and raises it to a specified power.
Prepopulating Value	A value option which is set in a form field prior to a user filling it out.
Property	A characteristic of an object within JavaScript.
Push	An array concept which is used to add an item to an array.
RNG	A random number generator (RNG) is used to add the randomly generated numbers to a game, creating the element of surprise.
Round	A method used in JavaScript to round a number to the nearest whole number.
Scope	An item within JavaScript which defines the depth to where a variable can be used in a file. Two factors determine scope: where the variable is declared and whether it is explicitly declared, meaning whether the var keyword is being used.
Shift	A method which is used to remove the first item from an array.
Single-Dimensional Array	An array with a single object that only has one group of values.

Sqrt	A type of method which calculates the square root of a number.
String	One of the most common variables in JavaScript, it is used to store text, numbers, symbols, and other characters.
Switch	A conditional statement which compares code against multiple cases and executes one or more blocks of code based on matching cases.
Try/Catch	A block in which code tries to run and, if there is an error, the catch part controls what happens next.
typeof Operator	A tool used in JavaScript which reveals the data type that is assigned to a variable.
Undefined	An empty variable which had been declared but has no value.
Unshift	An array method which is used to add an item to the beginning of an array.
Use Strict	A command used to force every variable in a function to be explicitly declared as a variable.
Validation	The act of making sure valid data is entered on a form before the form is submitted. The two most common validation tasks are to make sure required fields are filled out and to make sure the correct type of data is entered into a field.
Variable	A container which is used to hold information in JavaScript that can be used later in the code.
While Loop	A loop which proposes an argument where if something is true or false it will run a designated code.
Window	An object which represents a browser window in JavaScript.

JavaScript Lesson Plan

Each lesson is approximately 30-35 minutes and includes video training, fill-in-the-blanks, labs, projects, tests, as well as the course support files to follow along with the instructor. There is also a Notes section for customizable lesson plans and extra notes.

By adhering to the lesson plan, it will take approximately 28.5 class hours to complete the course. For workbook projects, we recommend students do the fill-in-the-blank sections as they watch the course.

Domain I: 11 lessons for approximately 6.5 hours

Day	Tasks	What Is Covered	Files	Notes
1	Pre-Assessment Videos: Introduction Course Opener How to Study for This Exam Assignment and Arithmetic Operators Assignment Increment and Decrement Workbook projects: Fill-in-the-Blanks Domain I Items 1-2 Using Assignment, Increment, and Decrement Operators	I JavaScript Operators, Methods, and Keywords I.1 Complete and debug code that uses assignment and arithmetic operators I.1.a Assignment I.1.b Increment I.1.c Decrement	I la-cartassignments.htm	
2	Videos: Assignment and Arithmetic Operators Addition Subtraction Division Multiplication Modulus Workbook projects: Fill-in-the-Blanks Domain I Items 3-7 Addition and Subtraction Multiplication, Division, and Modulus Operators	I JavaScript Operators, Methods, and Keywords I.1 Complete and debug code that uses assignment and arithmetic operators I.1.d Addition I.1.e Subtraction I.1.f Division I.1.g Multiplication I.1.h Modulus	I ld-cart-operators.htm I lf-soundproofing.htm	
3	Videos: Assignment and Arithmetic Operators Compound Assignment Operators JavaScript Best Practices Comments Indentations Lab: Compound Assignment Operator Symbols Workbook projects: Fill-in-the-Blanks Domain I Items 8-10 Compound Assignment Operators	I JavaScript Operators, Methods, and Keywords I.1 Complete and debug code that uses assignment and arithmetic operators I.1.i Compound assignment operators (+=, -=, *=, /=, %=) I.2 Apply JavaScript best practices I.2.a Comments I.2.b Indentations	I li-cart-compound.htm	

Day	Tasks	What Is Covered	Files	Notes
4	Videos: - JavaScript Best Practices - Naming Conventions - Noscript - Constants Workbook projects: - Fill-in-the-Blanks Domain I Items 11-13 - Comments, Indentations, and Naming Conventions - Using Script and Noscript Tags	I JavaScript Operators, Methods, and Keywords I.2 Apply JavaScript best practices - I.2.a Comments - I.2.b Indentations - I.2.c Naming conventions - I.2.d Noscript - I.2.e Constants	I2a-soundproofing.htm I2d-tuningkits.htm	
5	Videos: - JavaScript Best Practices - Reserved Keywords - Debugger Keyword Workbook projects: - Fill-in-the-Blanks Domain I Items 14-15 - Constants and Reserved Keywords - The Debugger Keyword	I JavaScript Operators, Methods, and Keywords I.2 Apply JavaScript best practices - I.2.e Constants - I.2.f Reserved keywords - I.2.g Debugger keyword	I2e-cart-constants.htm I2g-cart-constants.htm	
6	Videos: - JavaScript Best Practices - Set Breakpoints - Console.log - Inline and External Scripts - When to Use - How to Use Lab: - Code Sources Workbook projects: - Fill-in-the-Blanks Domain I Items 16-19 - Using Breakpoints and Console.log	I JavaScript Operators, Methods, and Keywords I.2 Apply JavaScript best practices - I.2.h Setting breakpoints - I.2.i Console.log I.3 Evaluate the use of internal and external scripts - I.3.a When to use - I.3.b How to use	I2h-cart-constants.htm	
7	Videos: - Inline and External Scripts - Use Both Scripts - Exception Handling - Try and Catch Workbook projects: - Fill-in-the-Blanks Domain I Items 20-21 - Using Breakpoints and Console.log - The Use of Inline and External Scripts	I JavaScript Operators, Methods, and Keywords I.3 Evaluate the use of internal and external scripts - I.3.c What happens when scripts are used at multiple levels I.4 Implement exception handling - I.4.a Try - I.4.b Catch	I3a-cart-compound.htm main.js	
8	Videos: - Exception Handling - Finally - Browser Object Model Interaction - Manage State Lab: - Finally Block Workbook projects: - Fill-in-the-Blanks Domain I Items 22-23 - Implementing Exception Handling - Managing States	I JavaScript Operators, Methods, and Keywords I.4 Implement exception handling - I.4.a Try - I.4.b Catch - I.4.c Finally I.5 Complete and debug code that interacts with the Browser Object Model (BOM) - I.5.a Display dialogs	I4a-practice.htm I5a-cart-checkout.htm	

Day	Tasks	What Is Covered	Files	Notes
9	Videos: Browser Object Model Interaction Display Dialogs Determine Screen Size Domain I Exam Tips Workbook projects: Fill-in-the-Blanks Domain I Items 24-25 Display Dialogs Determining Screen Size	I JavaScript Operators, Methods, and Keywords I.5 Complete and debug code that interacts with the Browser Object Model (BOM) I.5.a Display dialogs I.5.b Determine screen size	I5b-cart-welcome.htm I5c-screensize.htm	
10	Review Post-Assessment			
11	Post-Assessment			

Domain 2: 18 lessons for approximately 10.5 hours

Day	Tasks	What Is Covered	Files	Notes
1	Pre-Assessment Videos: Primitive Data Types Number Boolean Lab: Boolean Variable Workbook project: Fill-in-the-blanks Domain 2 Items 1-2	2 Variables, Data Types, and Functions 2.1 Declare and use variables of primitive data types 2.1.a Number 2.1.b Boolean		
2	Videos: Primitive Data Types String Null Lab: Null Variable Workbook projects: Fill-in-the-blanks Domain 2 Items 3-4 Number, Boolean, and String Variables	2 Variables, Data Types, and Functions 2.1 Declare and use variables of primitive data types 2.1.a Number 2.1.b Boolean 2.1.c String 2.1.d Null	21a-cart-variables.htm	
3	Videos: Primitive Data Types Undefined TypeOf Operator Type Checking Functions Labs: TypeOf Type Functions Workbook projects: Fill-in-the-blanks Domain 2 Items 5-7 Null and Undefined Variables Type of Operators and Type Checking Functions	2 Variables, Data Types, and Functions 2.1 Declare and use variables of primitive data types 2.1.d Null 2.1.e Undefined 2.1.f Type of operator 2.1.g Type-checking functions	21d-cart-variables.htm 21f-cart-variables.htm	
4	Videos: Primitive Data Types Use Strict Convert Between Data Types Workbook projects: Fill-in-the-blanks Domain 2 Items 8-9 The Use Strict Phrase Converting Between Data Types	2 Variables, Data Types, and Functions 2.1 Declare and use variables of primitive data types 2.1.h Use strict 2.1.i Converting between data types (parseInt, parseFloat)	21h-practice-strict.htm 21i-cart-operators.htm	
5	Videos: Primitive Data Types Format Numbers String Operations Nesting Quotes Labs: Combining Data Workbook projects: Fill-in-the-blanks Domain 2 Items 10-12 Formatting Numbers String Operations	2 Variables, Data Types, and Functions 2.1 Declare and use variables of primitive data types 2.1.j Formatting numbers 2.1.k String operations 2.1.l Single quote vs. double quote (nesting)	21j-formatting-numbers.htm 21k-practice-strings.htm	

Day	Tasks	What Is Covered	Files	Notes
6	Videos: Primitive Data Types Initialization Declare and Use Arrays Build and Define an Array Iteration and Length Initialization Lab: Bulleted List Workbook projects: Fill-in-the-blanks Domain 2 Items 13-16 Initialization Build and Iterate Through Arrays	2 Variables, Data Types, and Functions 2.1 Declare and use variables of primitive data types 2.1.m Initialization 2.2 Declare and use arrays 2.2.a Single-dimensional arrays 2.2.c Iteration 2.2.d Initialization 2.2.e Define, sort, and search an array 2.2.g Length property	21m-cart-variables.htm 22a-tuningkits-array.htm	
7	Videos: Declare and Use Arrays Sorting and Searching Push and Pop Lab: Searching an Array Workbook projects: Fill-in-the-blanks Domain 2 Items 17-18 Sorting and Searching Push and Pop	2 Variables, Data Types, and Functions 2.2 Declare and use arrays 2.2.a Single-dimensional arrays 2.2.c Iteration 2.2.d Initialization 2.2.e Define, sort, and search an array 2.2.f Push, pop, shift, and unshift methods	22e-tuningkits-array.htm 22f-tuningkits-array.htm	
8	Videos: Declare and Use Arrays Shift and Unshift Accessing an Element Understanding Multi-Dimensional Arrays Lab: Element Access Workbook projects: Fill-in-the-blanks Domain 2 Items 19-21 Shift and Unshift Access an Element Using Multi-Dimensional Arrays	2 Variables, Data Types, and Functions 2.2 Declare and use arrays 2.2.b Multi-dimensional arrays 2.2.f push, pop, shift, and unshift methods 2.2.g Length property 2.2.h Access an array element	22g-tuningkits-array.htm 22h-tuningkits-array.htm 22j-tuningkits-array.htm	
9	Videos: Complete and Debug Code on Objects Properties Methods Instantiation Workbook projects: Fill-in-the-blanks Domain 2 Items 22-24 Properties and Methods Instantiation	2 Variables, Data Types, and Functions 2.3 Complete and debug code that uses objects 2.3.a Properties 2.3.b Methods 2.3.c Instantiation	23a-cart-constants.htm 23c-products-class.htm main.js	

Day	Tasks	What Is Covered	Files	Notes
10	Videos: - Complete and Debug Code on Objects - Date Object and Time - Retrieving Date Parts Lab: - Apply the Date Workbook projects: - Fill-in-the-blanks Domain 2 Items 25-26 - Date Object and Time Retrieving Date Parts	2 Variables, Data Types, and Functions 2.3 Complete and debug code that uses objects 2.3.d Date object 2.3.e Retrieve date and time parts	23d-practice-dates.htm 23e-practice-dates.htm	
11	Videos: - Complete and Debug Code on Objects - Localization - Add and Subtract Dates Labs: - Date Localization - Adding Dates Workbook projects: - Fill-in-the-blanks Domain 2 Items 27-28 - Localization - Add and Subtract Dates	2 Variables, Data Types, and Functions 2.3 Complete and debug code that uses objects 2.3.f Localize date format (MM/DD vs DD/MM) 2.3.g Add and subtract dates	23f-practice-dates.htm 23g-practice-dates.htm	
12	Videos: - Built-In Math Functions - Random - Round - Abs Labs: - Random Number Generator - Round Method Workbook projects: - Fill-in-the-blanks Domain 2 Items 29-31 - Random Numbers - Rounding and Absolute Methods	2 Variables, Data Types, and Functions 2.4 Complete and debug code that uses built-in Math functions 2.4.a Random 2.4.b Round 2.4.c Abs	24a-jukebox.htm Blues.mp4 Happy.mp4 24b-practice-numbers.htm	
13	Videos: - Built-In Math Functions - Floor and Ceiling - Min and Max - Pow Labs: - Ceiling Method Workbook projects: - Fill-in-the-blanks Domain 2 Items 32-34 - Floor and Ceiling - Min and Max	2 Variables, Data Types, and Functions 2.4 Complete and debug code that uses built-in Math functions 2.4.d Floor 2.4.e Ceil 2.4.f Min 2.4.g Max 2.4.h Pow	24d-practice-numbers.htm 24f-cart-max-min.htm	

Day	Tasks	What Is Covered	Files	Notes
14	Videos: - Built-In Math Functions - Sqrt - Parameters and Return Values - Reusable Code - Local vs. Global Scope Lab: - Sqrt Syntax Workbook projects: - Fill-in-the-blanks Domain 2 Items 35-37 - Powers and Square Roots - Reusable Code	2 Variables, Data Types, and Functions 2.4 Complete and debug code that uses built-in Math functions 2.4.h Pow 2.4.i Sqrt 2.5 Complete and debug functions that accept parameters and return values 2.5.a Reusable code 2.5.b Local vs. global scope	24g-square-footage.htm 25a-cart-numbers.htm	
15	Videos: - Parameters and Return Values - Redefining Variables - Passing Parameters - Value vs. Reference Lab: - Defining a Parameter Workbook projects: - Fill-in-the-blanks Domain 2 Items 38-40 - Scopes and Variable Definitions - Value vs. Reference	2 Variables, Data Types, and Functions 2.5 Complete and debug functions that accept parameters and return values 2.5.b Local vs. global scope 2.5.c Redefine variables 2.5.d Pass parameters 2.5.e Value vs. reference	25b-cart-scopes.htm 25b-main.js 25e-cart-numbers.htm	
16	Videos: - Parameters and Return Values - Return Values - Domain 2 Exam Tips Workbook projects: - Fill-in-the-blanks Domain 2 Item 41 - Return Values - Review Domain 2	2 Variables, Data Types, and Functions 2.5 Complete and debug functions that accept parameters and return values 2.5.f Return values	25f-practice-numbers.htm	
17	Post-Assessment			
18	Post-Assessment			

Domain 3: 8 lessons for approximately 4.5 hours

Day	Tasks	What Is Covered	Files	Notes
1	Pre-Assessment Videos: Logical and Comparison Operators Equals Not Equals Greater Than and Less Than Workbook projects: Fill-in-the-blanks Domain 3 Items 1-3 Equals and Not Equals Greater Than or Less Than	3 Decisions and Loops 3.1 Evaluate expressions that use logical and comparison operators 3.1.a != 3.1.b < 3.1.c > 3.1.g ==	31a-cart-evaluation.htm 31c-practice-comparison.htm	
2	Videos: Logical and Comparison Operators Less Than or Equal To Greater Than or Equal To Not Workbook projects: Fill-in-the-blanks Domain 3 Items 4-6 Other Comparison Operators Not	3 Decisions and Loops 3.1 Evaluate expressions that use logical and comparison operators 3.1.d <= 3.1.e >= 3.1.f !	31d-practice-comparison.htm 31f-cart-evaluation.htm	
3	Videos: Logical and Comparison Operator And Or Lab: Testing the And Operator Workbook projects: Fill-in-the-blanks Domain 3 Items 7-8 And vs. Or	3 Decisions and Loops 3.1 Evaluate expressions that use logical and comparison operators 3.1.h && 3.1.i	31g-tuningkits-andor.htm	
4	Videos: Decision Statements If Else If Lab: Else If Statement Workbook projects: Fill-in-the-blanks Domain 3 Items 9-10 If-Else Else If	3 Decisions and Loops 3.2 Complete and debug decision statements 3.2.a Single alternative (if) 3.2.b Dual alternative (if else)	32a-cart-evaluation.htm 32b-tuningkits-andor.htm	
5	Videos: Decision Statements Switch Nested If Workbook projects: Fill-in-the-blanks Domain 3 Items 11-12 Switch Nested If	3 Decisions and Loops 3.2 Complete and debug decision statements 3.2.c Multiple alternative (switch) 3.2.d Nested if	32c-cart-switch.htm 32d-tuningkits-andor.htm	

Day	Tasks	What Is Covered	Files	Notes
6	Videos: Loops For While Do Break Workbook projects: Fill-in-the-blanks Domain 3 Items 13-16 For Do and While Loops	3 Decisions and Loops 3.3 Complete and debug loops 3.3.a For 3.3.b For in 3.3.c While 3.3.d Do while 3.3.e Break	33a-tuningkits-loops.htm 33b-practice-loops.htm	
7	Videos: Loops Continue Domain 3 Exam Tips Workbook projects: Fill-in-the-blanks Domain 3 Item 17 Break and Continue	3 Decisions and Loops 3.3 Complete and debug loops 3.3.e Break 3.3.f Continue	33d-practice-break.htm	
8	Post-Assessment			

Domain 4: 7 lessons for approximately 4 hours

Day	Tasks	What Is Covered	Files	Notes
1	Pre-Assessment Videos: DOM Tree Window Document Body Lab: Body Tag Workbook projects: Fill-in-the-Blanks Domain 4 Items 1-3 Window	4 Document Object Model 4.1 Identify and construct the DOM (Document Object Model) tree 4.1.a Window 4.1.b Document 4.1.c Body	41a-cart-DOM.htm	
2	Videos: DOM Tree Other HTML Elements HTML Events Onchange Workbook projects: Fill-in-the-Blanks Domain 4 Items 4-5 Document, Body and Other HTML Elements Onchange	4 Document Object Model 4.1 Identify and construct the DOM (Document Object Model) tree 4.1.b Document 4.1.c Body 4.1.d Other HTML elements 4.2 Identify and handle document, form, keyboard, and mouse events 4.2.d Onchange	41b-tuningkits-DOM.htm 42a-practice-DOM.htm	
3	Videos: HTML Events Onmouseover Onload Labs: Onmouseover Event Onload Event Workbook projects: Fill-in-the-Blanks Domain 4 Items 6-7 Onmouseover	4 Document Object Model 4.2 Identify and handle document, form, keyboard, and mouse events 4.2.a Onload 4.2.d Onchange 4.2.i Onmouseover	42b-products-DOM.htm	
4	Videos: HTML Events Onclick Onmouseout Onkeydown Workbook projects: Fill-in-the-Blanks Domain 4 Items 8-10 Onload and Onclick Onmouseout Onkeydown	4 Document Object Model 4.2 Identify and handle document, form, keyboard, and mouse events 4.2.a Onload 4.2.e Onkeydown 4.2.h Onclick 4.2.j Onmouseout	42c-cart-DOM.htm 42e-cart-mouse.htm 42f-cart-mouse.htm	

Day	Tasks	What Is Covered	Files	Notes
5	Videos: - Output to HTML Documents - InnerHTML - Document.write - Locate, Modify, and Add Elements - GetElementById Lab: - Append InnerHTML Workbook projects: - Fill-in-the-Blanks Domain 4 - Items 11-13 - InnerHTML - Document.write	4 Document Object Model 4.3 Complete and debug code that outputs to an HTML document 4.3.a Document.write 4.3.b InnerHTML 4.4 Complete and debug code that locates, modifies, and adds HTML elements and attributes to documents 4.4.a GetElementById	43a-practice-DOM.htm 43b-cart-DOM.htm	
6	Videos: - Locate, Modify, and Add Elements - GetElementsByTagName - GetElementsByClassName - SetAttribute - CreateElement - Domain 4 Exam Tips Lab: - GetElementsByClassName Method Workbook projects: - Fill-in-the-Blanks Domain 4 - Items 14-17 - GetElement - SetAttribute and CreateElement	4 Document Object Model 4.4 Complete and debug code that locates, modifies, and adds HTML elements and attributes to documents 4.4.a GetElementById 4.4.b GetElementsByTagName 4.4.c GetElementsByClassName 4.4.d SetAttribute 4.4.e CreateElement 4.5 Create events using event handlers and listeners 4.5.a DOM events 4.5.b HTML attribute event 4.5.c AddEventListener	44a-comparison-get.htm 44d-comparison-get.htm	
7	Post-Assessment			

Domain 5: 5 lessons for approximately 3 hours

Day	Tasks	What Is Covered	Files	Notes
1	Pre-Assessment Videos: Retrieve Input and Set Form Field Values Retrieve Form Values DOM Path Values from Elements Workbook projects: Fill-in-the-blanks Domain 5 Items 1-3 Retrieving Form Values DOM Path Values from Elements	5 HTML Forms 5.1 Complete and debug code that retrieves form input and sets form field values 5.1.a Retrieve form values 5.1.b Identify the DOM path 5.1.c Get values from different types of elements	51a-contact.htm 51b-contact.htm 51c-contact.htm	
2	Videos: Retrieve Input and Set Form Field Values Prepopulating Values Masking Values Input Validation Upper and Lower Case Lab: Understanding Prepopulating Code Workbook projects: Fill-in-the-blanks Domain 5 Items 4-6 Prepopulating Values Masking Values	5 HTML Forms 5.1 Complete and debug code that retrieves input from forms and sets form field values 5.1.d Prepopulate values 5.1.e Mask values 5.2 Complete and debug code that performs input validation 5.2.a Case	51d-contact.htm 51e-contact.htm	
3	Videos: Input Validation String Comparisons Validation Workbook projects: Fill-in-the-blanks Domain 5 Items 7-8 Uppercase and Lowercase Validation	5 HTML Forms 5.2 Complete and debug code that performs input validation 5.2.a Case 5.2.b String comparisons 5.2.d Not blank	52a-contact.htm 52c-contact.htm	
4	Videos: Input Validation NaN Form Submission Process Onsubmit Understand Post vs. Get Understand Potential Submission Targets Lab: Targets Workbook projects: Fill-in-the-blanks Domain 5 Items 9-12 NaN Onsubmit Post vs. Get and Submission Targets	5 HTML Forms 5.2 Complete and debug code that performs input validation 5.2.c NaN 5.3 Describe the form submission process 5.3.a Onsubmit 5.3.b Post vs. get 5.3.c Potential targets for submission	52d-practice-DOM.htm 53a-contact.htm 53b-contact.htm	

Day	Tasks	What Is Covered	Files	Notes
5	Videos: Form Submission Process Domain 5 Exam Tips Conclusion Final Exam Tips Final Thoughts Post-Assessment			

JavaScript Domain I Outline

Introduction

- Course Opener
- How to Study for This Exam

Assignment and Arithmetic Operators

- Assignment
- Increment and Decrement
- Addition
- Subtraction
- Division
- Multiplication
- Modulus
- Compound Assignment Operators

JavaScript Best Practices

- Comments
- Indentations
- Naming Conventions
- Noscript
- Constants
- Reserved Keywords
- Debugger Keyword
- Set Breakpoints
- Console.log

Inline and External Scripts

- When to Use
- How to Use
- Use Both Scripts

Exception Handling

- Try and Catch
- Finally

Browser Object Model Interaction

- Manage State
- Display Dialogs
- Determine Screen Size
- Domain I Exam Tips

JavaScript Domain 2 Outline

Primitive Data Types

- Number
- Boolean
- String
- Null
- Undefined
- TypeOf Operator
- Type Checking Functions
- Use Strict
- Convert Between Data Types
- Format Numbers
- String Operations
- Nesting Quotes
- Initialization

Parameters and Return Values

- Reusable Code
- Local vs. Global Scope
- Redefining Variables
- Passing Parameters
- Value vs. Reference
- Return Values
- Domain 2 Exam Tips

Declare and Use Arrays

- Build and Define an Array
- Iteration and Length
- Initialization
- Sorting and Searching
- Push and Pop
- Shift and Unshift
- Access an Element
- Understand Multi-Dimensional Arrays

Complete and Debug Code on Objects

- Properties
- Methods
- Instantiation
- Date Object and Time
- Retrieving Date Parts
- Localization
- Add and Subtract Dates

Built-In Math Functions

- Random
- Round
- Abs
- Floor and Ceiling
- Min and Max
- Pow
- Sqrt

JavaScript Domain 3 Outline

Logical and Comparison Operators

- Equals
- Not Equals
- Greater Than and Less Than
- Less Than or Equal To
- Greater Than or Equal To
- Not
- And
- Or

Decision Statements

- If
- Else If
- Switch
- Nested If

Loops

- For
- While
- Do
- Break
- Continue
- Domain 3 Exam Tips

JavaScript Domain 4 Outline

DOM Tree

- Window
- Document
- Body
- Other HTML Elements

HTML Events

- Onchange
- Onmouseover
- Onload
- OnClick
- Onmouseout
- Onkeydown

Output to HTML Documents

- InnerHTML
- Document.Write

Locate, Modify, and Add Elements

- GetElementById
- GetElementsByTagName
- GetElementsByClassName
- SetAttribute
- CreateElement
- Domain 4 Exam Tips

JavaScript Domain 5 Outline

Retrieve Input and Set Form Field Values

- Retrieve Form Values
- DOM Path
- Values from Elements
- Prepopulating Values
- Masking Values

Input Validation

- Upper and Lower Case
- String Comparisons
- Validation
- NaN

Form Submission Process

- Onsubmit
- Understand Post vs. Get
- Understand Potential Submission Targets
- Domain 5 Exam Tips

Conclusion

- Final Exam Tips
- Final Thoughts

